

Python Interview Questions and Answers

From absolute beginner to experienced engineer

Python interviews change as your responsibilities grow. At the beginning, you need to explain what a program does and write correct functions. Later, you need to build reliable APIs, understand database behavior, diagnose failures, and make decisions that other engineers can maintain. At the senior level, the important question becomes whether your design remains correct when requests overlap, dependencies slow down, deployments fail, and requirements change.

This guide follows that progression. It starts with programming vocabulary, then moves through the Python language, coding interviews, web frameworks, databases, testing, concurrency, infrastructure, data engineering, AI applications, and system design. Each numbered question has an answer. The longer worked exercises show how to turn an answer into code or an engineering decision.

Experience ranges are reading suggestions, not rigid hiring rules. Someone with two years of focused backend work may understand a topic better than someone with ten years in another specialty. Use the questions to find gaps, then practice explaining your reasoning without reading the answer.

Technical baseline: core examples target Python 3.11 or newer unless explicitly marked. Version notes discuss Python 3.14, Django 5.2, Flask 3.x, Pydantic v2, SQLAlchemy 2.x, and pandas 3.x. These are explicit reference versions, not a claim that every example applies unchanged to every release. Framework examples marked as fragments require an existing application and dependencies. Source checks were performed on September 15, 2026. Official references appear at the end; bracketed reference numbers identify particular version-sensitive claims.

How to use this guide

For each question, give a short answer first. Then explain one example, one failure case, and one trade-off. For coding exercises, state the input contract, write a simple correct approach, improve it when justified, and test boundaries. For system design, agree on requirements before naming tools.

Beginner route: Chapters 1 through 7, then the first coding exercises and testing basics. Junior route: add object models, typing, APIs, SQL, and one framework. Mid-level route: add concurrency, queues, security, profiling, and deployment. Senior route: study all system design cases, migration scenarios, operational decisions, and leadership questions; return to basic language questions whenever your explanation is uncertain.

The document is also organized as a long-form blog series. Each chapter can become one article. The explanations are written in plain English, and the code examples are deliberately small enough to discuss during an interview. They are learning examples, not complete production applications.

Contents

- 1 Starting from absolute beginner
- 2 Numbers strings and built in collections
- 3 Identity mutation and function behavior
- 4 Iteration generators exceptions and files
- 5 Object oriented Python and the data model
- 6 Typing imports packaging and maintainability
- 7 Complexity and coding interview patterns
- 8 Concurrency parallelism and Python internals
- 9 Testing debugging and code review
- 10 HTTP APIs authentication and web architecture
- 11 Django interview questions
- 12 Django REST Framework interview questions
- 13 Flask interview questions
- 14 FastAPI Pydantic and async APIs
- 15 SQL databases and SQLAlchemy
- 16 Caching queues and distributed workflows
- 17 Security reliability and deployment
- 18 Data engineering numerical Python and AI applications
- 19 Senior engineering and architectural judgment
- 20 Recent Python changes and difficult follow ups
- 21 Worked coding exercises with solutions
- 22 Worked Python system design interviews
- 23 Production debugging interview scenarios
- 24 Practice plans mock interviews and assessment
- 25 Official references and further study

1 Starting from absolute beginner

Q001 What is programming and where does Python fit

Programming means writing instructions that a computer can execute. A program receives input, transforms or stores it, and produces output or effects. Python is a general-purpose language used for web services, automation, data processing, testing, scientific computing, and many other tasks. It provides readable syntax and a large ecosystem of libraries.

An interview answer should distinguish a language from its runtime and libraries. Python is the language; CPython is its most common implementation; Django is a web framework written for Python. Installing Django does not create a new programming language.

Q002 What are a variable an object and a type

An object is a value with behavior and a type. A variable name refers to an object. In `age = 25`, the name `age` refers to an integer object. Assigning `age = "twenty five"` later binds the same name to a string; it does not transform the old integer object into a string.

Types determine valid operations. Adding two integers performs arithmetic, while adding two strings concatenates them. Thinking in terms of names referring to objects will help you understand mutation, function arguments, copying, and shared state.

Q003 How do you run Python code

You can enter statements in an interactive interpreter, run a script such as `python interview.py`, or use an editor or notebook that starts an interpreter for you. A script is useful for repeatable execution. The interactive interpreter is useful for quick experiments. A notebook combines code, output, and explanation but can hide dependencies on cells executed earlier.

When a command behaves differently across environments, inspect which interpreter is running. A package installed for one interpreter may be unavailable to another. In project instructions, prefer `python -m pip` to help associate package installation with the selected interpreter.

Q004 Why does indentation matter

Indentation defines blocks such as function bodies, loops, and conditional branches. Consistent indentation is syntax, not decoration. Use four spaces per level as the usual style and avoid mixing tabs and spaces. An incorrectly indented statement may either fail to parse or run in the wrong branch.

```
score = 72
if score >= 60:
    result = "pass"
else:
    result = "retry"
print(result) # pass
```

Q005 What are input output and type conversion

`input()` reads text. Even if the user enters 42, the result is a string until converted. `int(text)` parses an integer or raises an exception when the text is invalid. `print()` produces a human-readable representation, while a function's return gives a value back to its caller.

Do not confuse printing a result with returning it. Automated tests and other functions generally need returned values. Keep input parsing separate from the calculation so you can test the calculation without simulating a keyboard.

Q006 What is the difference between syntax runtime and logic errors

A syntax error means the code cannot be parsed. A runtime error happens during execution, such as dividing by zero or indexing beyond a list. A logic error produces the wrong result even though execution finishes.

For example, `return total / len(values)` fails at runtime for an empty list. Returning `total / 10` for every input may run but is logically wrong. Tests should check ordinary inputs and boundaries because successful execution alone does not demonstrate correctness.

Q007 How do conditions and Boolean operators work

Conditions choose a branch based on truthiness. and stops when it finds a false-like operand; or stops when it finds a true-like operand. They return operands rather than always returning True or False. `not` returns a Boolean.

For example, `name or "anonymous"` chooses a fallback for any false-like value, including the empty string. If only None means missing, use an explicit `is None` check. Otherwise a valid value such as numeric zero can accidentally be replaced by a default.

Q008 How do for and while loops differ

A for loop consumes an iterable, such as a list, a range, or a file. A while loop repeats while a condition is true. Prefer for when you are visiting items and while when the number of iterations depends on changing state.

`break` exits the loop; `continue` skips the remainder of the current iteration. A loop's `else` block runs when the loop finishes without `break`, including when there were no iterations. It is useful for searches but deserves an explanation because readers may confuse it with an `if` branch.

Q009 What does range produce

`range(start, stop, step)` describes an immutable arithmetic sequence, with the stop value excluded. `range(3)` represents 0, 1, 2. It does not build a list of all those integers. The step cannot be zero, and a negative step is used to move downward.

`range` is iterable but is not a generator. It supports operations such as `length` and `indexing`. This distinction matters when an interviewer asks you to compare an iterable, iterator, and generator rather than treating all lazy-looking objects as equivalent.

Q010 What should a beginner explain before writing code

Explain what the inputs mean, what should be returned, and how invalid or empty inputs are handled. Ask whether duplicate values, negative numbers, Unicode, or huge inputs are possible when they affect the solution. Walk through a small example by hand.

For a maximum-average question, agree that the window has exactly `k` items and that `1 <= k <= len(values)`. That single clarification prevents incorrect handling of zero division, empty windows, and partially filled windows.

2 Numbers strings and built in collections

Q011 Which built in types should you know first

Learn int, float, bool, str, list, tuple, dict, set, and None. Add bytes, bytearray, frozenset, and complex as you encounter their use cases. The important distinctions are whether values are mutable, ordered, indexed, unique, and hashable.

Do not memorize only names. Be able to choose a list for ordered records, a set for membership, and a dictionary for lookup by a key. An appropriate data structure often matters more than a clever loop.

Q012 How are integers and floats different

Python integers support arbitrary precision within available memory. Floats normally use binary floating-point, so many decimal fractions cannot be represented exactly. For example, `0.1 + 0.2 == 0.3` is false because of representation and rounding.

Use `math.isclose` with tolerances appropriate to the problem when comparing approximate numerical results. Use integer minor units or carefully configured Decimal arithmetic when exact decimal behavior is required. Construct a decimal from a string to avoid importing an already rounded binary float.

Q013 What do division floor division and modulo mean

`/` performs true division. `//` rounds the quotient toward negative infinity. `%` gives the corresponding remainder, so for nonzero `b`, `a == (a // b) * b + a % b`. With a positive divisor, the remainder is nonnegative.

For example, `-7 // 3` is `-3` and `-7 % 3` is `2`. Floor division is not the same as truncating toward zero. This matters when porting algorithms from languages with different integer division rules.

Q014 Why are strings immutable

A string's contents cannot be changed in place. An operation such as `text.replace("a", "b")` returns a string rather than modifying the original. You can rebind a name to the returned string, but that is a different operation from mutation.

For assembling many pieces, collect them and use `"".join(pieces)` rather than depending on repeated concatenation behavior. Some implementations optimize certain concatenation cases, but an algorithm should not rely on that optimization for predictable scaling.

Q015 What is the difference between str and bytes

`str` represents Unicode text. `bytes` represents byte values. Encoding converts text to bytes; decoding converts bytes to text using an agreed encoding such as UTF-8. Network and file boundaries are where this distinction becomes important.

```
text = "café"
payload = text.encode("utf-8")
assert payload.decode("utf-8") == text
```

Do not repeatedly encode and decode without knowing what each function expects. Decide how invalid byte sequences should be handled instead of silently discarding data.

Q016 Does a string index always represent one visible character

No. Python string indexing works with Unicode code points, while one user-visible character may contain multiple code points. Accented text can also have more than one equivalent representation. Emoji can contain multiple code points joined into one visible symbol.

A basic interview palindrome question often assumes simple character comparisons. A production feature may require Unicode normalization, case folding, or grapheme segmentation. State the contract instead of claiming that reversing code points always reverses displayed characters correctly.

Q017 How do lists and tuples differ

A list is a mutable sequence. A tuple is an immutable sequence of references. A tuple may still contain a mutable object, so `([1],)` can contain a list that later changes. Tuple immutability does not recursively freeze its contents.

Use a list when membership or order will change. Use a tuple for a fixed grouping or immutable sequence when that meaning helps readers. A tuple is hashable only if all of its elements are hashable; placing a list inside it prevents using it as a dictionary key.

Q018 How do append extend and insert differ

`append(x)` adds one item at the end. `extend(items)` adds every item from an iterable. `insert(i, x)` inserts at a position and may shift subsequent items. These mutating methods return `None`; assigning their result is a common bug.

```
values = [1]
values.append([2, 3]) # [1, [2, 3]]
values.extend([4, 5]) # [1, [2, 3], 4, 5]
```

Appending is amortized constant time for the usual CPython list implementation. Inserting near the beginning is linear because existing references need to move.

Q019 What is slicing and what does it cost

`items[start:stop:step]` selects a portion of a sequence. Stop is excluded. Negative indices count from the end, and omitted bounds use defaults. `items[::-1]` reverses a sequence through slicing.

For a list, a slice creates a new list containing references to the selected objects. A slice of length `k` generally costs $O(k)$ time and $O(k)$ extra space. Repeated slicing inside recursion can turn a seemingly efficient algorithm into one with substantial copying overhead.

Q020 How do dictionaries work conceptually

A dictionary maps hashable keys to values. Hashing helps locate candidate entries, and equality distinguishes matching keys from collisions. Lookups, insertions, and deletions are normally average $O(1)$, but worst-case behavior and the cost of hashing or equality still matter.

Modern Python dictionaries preserve insertion order. That does not mean keys are automatically sorted. Updating an existing key does not create a new position, while removing and reinserting a key changes its insertion position.

Q021 How do you handle a missing dictionary key

Use indexing when a missing key is an error, `.get()` when a default is meaningful, and `defaultdict` when creating a default on first access is intentional. `setdefault` can both retrieve and insert a value. Remember that function arguments are evaluated before the call, so an expensive default expression can run even when the key exists.

Distinguish a missing key from a key whose value is `None`. A unique sentinel object is helpful when `None` is a legitimate stored value.

Q022 When is a set better than a list

A set is useful for uniqueness and repeated membership checks. If you repeatedly ask whether an identifier is present, building a set can reduce repeated linear scans to average constant-time lookups. Sets do not provide sequence indexing or a guaranteed iteration order.

The conversion has a cost, so it may not help for a single lookup in a tiny list. For order-preserving deduplication of hashable values, `list(dict.fromkeys(values))` keeps the first occurrence. Clarify whether equality-based deduplication matches the business rules.

Q023 What makes an object hashable

A hashable object has a hash value that remains stable over its lifetime and an equality relationship consistent with hashing: equal objects must have equal hashes. The reverse is not required because collisions are allowed.

Do not equate hashable with simply immutable. Some user-defined mutable objects use identity-based equality and hashing. The dangerous case is changing fields used by a custom hash after inserting the object into a dictionary or set, because future lookup behavior can become incorrect.

Q024 What collections are useful beyond list and dict

`Counter` counts hashable items. `defaultdict` supplies missing values through a factory. `deque` supports efficient operations at both ends. `namedtuple` provides tuple behavior with field names. A heap maintained with `heapq` supports priority selection.

Choose by operations rather than appearance. A queue implemented with `list.pop(0)` shifts remaining elements. A `deque.popleft()` avoids that repeated shifting. For a bounded history, a deque with `maxlen` automatically discards old entries as new ones arrive.

Q025 How do sorting functions differ

`sorted(iterable)` returns a new list. `list.sort()` mutates a list and returns `None`. Both accept a key function and a reverse flag. Python sorting is stable, meaning items with equal keys retain their relative order.

```
people = [("Maya", 30), ("Arun", 25), ("Nora", 30)]
ordered = sorted(people, key=lambda person: person[1])
assert ordered == [("Arun", 25), ("Maya", 30), ("Nora", 30)]
```

Explain what happens on ties. A secondary key such as `(age, name)` makes the intended order explicit.

Q026 What are enumerate zip any and all useful for

enumerate supplies positions alongside items. zip pairs iterables and normally stops at the shortest; strict=True can detect different lengths. any stops at the first true-like item, while all stops at the first false-like item.

On an empty iterable, any returns false and all returns true. Those results are mathematically useful but can surprise validation code. If a form requires at least one selected item, all(valid(x) for x in items) alone does not enforce nonemptiness.

3 Identity mutation and function behavior

Q027 What is the difference between equality and identity

== asks whether objects compare equal. is asks whether they are the same object. Use is None for the singleton None, and use equality for ordinary value comparisons. Two separately created lists can compare equal without being identical.

Do not use observed integer caching or string interning as a reason to compare values with is. Those implementation optimizations can vary by construction and runtime. A correct answer should not depend on a particular interactive interpreter experiment.

Q028 Does assigning a list copy it

No. second = first gives another name to the same list. Mutation through either name is visible through the other. To create a shallow copy, use first.copy() or first[:].

```
first = [1, 2]
second = first
second.append(3)
assert first == [1, 2, 3]
assert first is second
```

This behavior is often the root cause of unexpected changes to configuration dictionaries, cached values, or objects passed between functions.

Q029 What is shallow copy versus deep copy

A shallow copy creates a new outer container while retaining references to nested objects. A deep copy recursively copies supported nested objects while tracking already visited objects. Deep copying is not automatically appropriate for database sessions, file handles, locks, or objects with external resources.

```
import copy
original = [[1], [2]]
shallow = copy.copy(original)
deep = copy.deepcopy(original)
original[0].append(9)
assert shallow[0] == [1, 9]
assert deep[0] == [1]
```

Before copying a large structure, ask whether immutable values or clear ownership would solve the problem with less cost.

Q030 How are arguments passed to Python functions

Arguments bind local parameter names to the objects supplied by the caller. Rebinding a parameter does not rebind the caller's variable. Mutating a shared mutable object is visible to the caller. This is often described as call by sharing or object-reference assignment.

```
def change(items):
    items.append(2)
    items = [99]

data = [1]
change(data)
assert data == [1, 2]
```

Avoid answering only “pass by value” or “pass by reference” without explaining the observable behavior; those phrases mean different things across languages.

Q031 Why are mutable default arguments dangerous

Default expressions are evaluated when the function is defined, not freshly on each call. A list default can therefore retain values between calls. Use None or a sentinel and allocate inside the function when each invocation needs a fresh object.

```
def collect(value, items=None):
    if items is None:
        items = []
    items.append(value)
    return items

assert collect(1) == [1]
assert collect(2) == [2]
```

Shared defaults can be intentional, but hidden state is rarely a good public API. Dataclasses similarly provide default_factory for per-instance mutable defaults.

Q032 What are positional keyword and keyword only parameters

Positional arguments are matched by position. Keyword arguments are matched by parameter name. A / in a signature makes preceding parameters positional-only; a * makes subsequent parameters keyword-only. These markers communicate how callers should use an API.

```
def connect(host, /, *, timeout=5):
    return host, timeout

assert connect("db.internal", timeout=2) == ("db.internal", 2)
```

Keyword-only options make calls with several booleans or timeouts easier to read. Positional-only parameters can preserve freedom to rename internal parameter names.

Q033 What are args and kwargs

*args collects extra positional arguments into a tuple. **kwargs collects extra keyword arguments into a dictionary. At a call site, unpacking expands an iterable or mapping into arguments. The names args and kwargs are conventions, not reserved keywords.

Use flexible signatures for wrappers or genuinely open-ended APIs. In business functions, explicit parameters provide clearer documentation and better type checking. Accepting every possible keyword can hide spelling errors until much later in execution.

Q034 What is scope and how do global and nonlocal work

Name lookup is commonly explained as local, enclosing function, global module, then builtins. Assignment usually makes a name local to the function unless declared otherwise. `global` refers to the module binding; `nonlocal` refers to an existing binding in an enclosing function scope.

A function that reads `count` and later assigns to it may raise `UnboundLocalError` because the assignment makes it local throughout that function body. Prefer explicit parameters and return values when shared mutable state is not necessary.

Q035 What is a closure and what is late binding

A closure retains access to variables in an enclosing function scope. Names are generally looked up when the inner function runs, which can surprise you when creating functions in a loop. Each function may see the same final loop variable.

```
functions = [lambda i=i: i for i in range(3)]
assert [f() for f in functions] == [0, 1, 2]
```

The default argument captures the current object at function creation. Another solution is a helper factory that creates a new enclosing scope for each value.

Q036 When should you use a lambda

A lambda creates a small anonymous function containing one expression. It works well for a short sorting key or callback. A named `def` is better when behavior needs multiple steps, documentation, repeated use, or a meaningful name in a traceback.

Lambda is not inherently faster or more functional than a normal function. Treat readability and debugging as design requirements. A dense lambda with several nested conditions often makes an interview solution harder to verify.

Q037 What does a decorator do

A decorator takes an object, commonly a function, and returns a replacement or modified object. `@decorate` above a function is broadly equivalent to rebinding its name to `decorate(function)`. Decorators can add logging, access checks, registration, or caching.

Use `functools.wraps` in function wrappers to preserve useful metadata and the wrapped-function link. An async function usually needs an async-aware wrapper. A synchronous wrapper that merely returns a coroutine does not measure its actual execution time or catch exceptions raised later during awaiting.

Q038 What is recursion and when does it become a problem

A recursive function solves a problem using smaller instances of the same problem. It needs a base case and progress toward that base case. Recursion can express trees and divide-and-conquer algorithms clearly, but deep recursion uses stack space and can exceed Python's recursion limit.

Python does not generally eliminate tail calls. For long linear traversals, use an iterative loop. For naive recursive Fibonacci, repeated subproblems cause exponential work; memoization or an iterative algorithm changes the complexity dramatically.

4 Iteration generators exceptions and files

Q039 What is an iterable versus an iterator

An iterable can provide an iterator through `iter(obj)`. An iterator provides successive items through `next()` and signals exhaustion with `StopIteration`. Lists are iterable and can provide fresh independent iterators. An iterator usually returns itself from `iter()` and maintains its current position.

This distinction explains why a generator passed through one loop may be empty in a later loop. If a function promises to consume an iterable only once, document that behavior; callers may provide a file or a generator rather than a reusable list.

Q040 How is a generator different from a list

A generator produces values as iteration advances and preserves its execution state between yields. A list stores its elements and supports indexing and repeated iteration. Generators can reduce peak memory when downstream code also processes incrementally.

Generators are not automatically faster. They have per-item execution overhead, and converting the result to a list removes the memory advantage. A generator can also retain references to large objects in its suspended frame, so “lazy” does not mean “uses no memory.”

Q041 What do `yield` and `yield from` mean

`yield` produces a value and pauses the generator. `yield from` delegates iteration to another iterable and also supports the generator protocol's richer interactions. For simple flattening, it avoids a manually written inner loop.

```
def flatten_once(groups):
    for group in groups:
        yield from group
```

```
assert list(flatten_once([[1, 2], [3]])) == [1, 2, 3]
```

This function flattens one level only. A recursive flattening function needs a contract for strings, mappings, cycles, and deeply nested input.

Q042 What makes comprehensions useful and when should you avoid them

A comprehension concisely constructs a collection from iteration and optional filtering. A generator expression uses parentheses and creates a generator instead of a list. Prefer comprehensions for simple transformations and filters.

Avoid side-effect comprehensions such as `[send(x) for x in items]` when the resulting list is unused. Use a loop. Also avoid deeply nested comprehensions when naming intermediate steps would make the transformation easier to test and explain.

Q043 How should exceptions be handled

Catch the narrowest exception that you can meaningfully handle. Keep the try block small enough that you know which operation failed. Use `else` for code that should run only when the protected operation succeeds and finally for cleanup that must run on exit.

Do not catch every exception and return a success-like value. That converts failures into corrupted assumptions. At application boundaries, translate internal failures into stable responses while recording appropriate diagnostic context without leaking secrets.

Q044 When should you define a custom exception

Define a custom exception when callers need to distinguish a domain-specific failure, such as `InsufficientInventory`, from unrelated implementation errors. Inherit ordinary application exceptions from `Exception`. Keep the hierarchy shallow and names meaningful.

Use exception chaining, such as `raise ImportError(...) from exc`, when translating a lower-level error and preserving its cause. Do not expose database connection strings or full sensitive input in the exception message that reaches a client.

Q045 What is a context manager

A context manager defines setup and cleanup around a `with` block. Files, locks, and database transaction scopes are common examples. The protocol uses `__enter__` and `__exit__`; `contextlib.contextmanager` lets you implement many cases with a generator and `try/finally`.

Returning a true-like value from `__exit__` suppresses an exception. Do this only when suppression is part of the contract. Resource cleanup and successful business completion are separate concerns: closing a file does not mean every intended write succeeded.

Q046 How do you process a large file safely

Open it with an explicit encoding for text, iterate rather than reading everything, and close it through a context manager. Consider maximum record size as well as total file size: one enormous line can still consume substantial memory.

```
def nonempty_lines(path):
    with open(path, encoding="utf-8") as stream:
        for line in stream:
            text = line.strip()
            if text:
                yield text
```

The file stays open while this generator is active. A caller that stops early should ensure the generator is closed or use an API with explicit resource ownership.

Q047 How do JSON CSV and pickle differ

JSON is a portable data format with a limited set of types. CSV represents tabular text but needs an agreed dialect, encoding, and schema. Pickle preserves many Python objects but is Python-specific and can execute code during deserialization.

Never unpickle untrusted data. For interchange, use an explicit schema and validate size and structure. JSON is safer than pickle for this purpose, but deeply nested or enormous JSON can still exhaust resources, and deserialized values still need validation.

Q048 What are common datetime mistakes

Mixing naive and timezone-aware datetimes, treating a local time as UTC, and assuming every day has the same local-clock structure cause bugs. Store and exchange clear timestamps, often in UTC, while retaining the business timezone when local-calendar rules matter.

Use a monotonic clock for elapsed durations and timeout calculations. A wall clock can jump because of clock corrections. For schedules such as “9 AM in New York,” a fixed UTC offset is insufficient because daylight saving rules can change the offset.

5 Object oriented Python and the data model

Q049 What are classes and instances

A class defines a kind of object and its behavior. An instance is a particular object created from that class. Instance attributes usually hold state belonging to one object, while class attributes are shared through class lookup unless shadowed.

Classes are useful when data and behavior belong together or when an object must satisfy a protocol. They are not mandatory for every problem. A small pure function may express a calculation more clearly than a class containing only one stateless method.

Q050 How do instance class and static methods differ

An instance method receives the instance as `self`. A class method receives the class as `cls` and is useful for alternate constructors that respect subclasses. A static method receives neither automatically and acts like a namespaced utility.

Do not choose `staticmethod` merely to avoid creating an instance if the function does not belong conceptually to the class. A module-level function may be simpler. In interviews, explain the binding behavior rather than saying that one method is universally faster.

Q051 What is the class attribute mutable state trap

If a mutable list is defined on the class and instances mutate it without creating their own list, they share the same object. Put per-instance mutable state in `__init__`, or use a `dataclass` field factory.

```
class Cart:
    def __init__(self):
        self.items = []

left, right = Cart(), Cart()
left.items.append("book")
assert right.items == []
```

Shared class state can be intentional for a registry, but it then needs a lifecycle and concurrency strategy.

Q052 What is inheritance versus composition

Inheritance models a subtype relationship and reuses behavior through a class hierarchy. Composition gives an object collaborators that provide behavior. Prefer composition when you need to replace a dependency, combine independent capabilities, or avoid coupling to a parent class's internal assumptions.

For example, a report service can receive a storage client rather than inherit from a storage implementation. The service remains responsible for report rules, and the storage object remains responsible for persistence. Inheritance remains useful when the framework or domain has a clear substitutable abstraction.

Q053 What are polymorphism and duck typing

Polymorphism lets the same operation work with different implementations. Duck typing focuses on supported behavior rather than a required inheritance relationship. A function that calls `stream.read()` can work with a file, an in-memory stream, or another compatible object.

Duck typing does not mean skipping contracts. Document required behavior, error cases, and ownership. Static Protocol definitions can make structural expectations visible to type checkers while retaining flexibility in the implementation.

Q054 What do encapsulation and private names mean in Python

A leading underscore signals an internal API by convention. Double-leading names inside classes trigger name mangling, mainly to reduce accidental collisions in subclasses. They are not a security boundary that prevents determined access.

Encapsulation means controlling how callers depend on implementation details. A property or method can enforce invariants, but adding getters and setters for every field does not automatically improve design. Use the smallest interface that protects meaningful rules.

Q055 What are properties and descriptors

A property gives method-backed access through attribute syntax. More generally, a descriptor defines attribute behavior through methods such as `__get__`, `__set__`, or `__delete__`. Descriptors support properties, bound methods, and many ORM field mechanisms.

For advanced interviews, know that data descriptors take precedence over an instance dictionary entry, while non-data descriptors can be shadowed by an instance attribute. Do not implement a descriptor when a simple property is enough. The Python data model is the detailed reference for this lookup behavior. [1]

Q056 What are `new` and `__init__` responsible for

`__new__` creates or returns the object; `__init__` initializes an instance after creation when appropriate. `__init__` must return `None`. Most ordinary classes only need `__init__`. Custom `__new__` is more relevant when subclassing immutable types or controlling creation.

Do not describe `__init__` as the entire object-allocation mechanism. In a senior interview, also explain why intercepting creation can complicate subclassing, testing, and serialization. A simple factory function may be clearer.

Q057 What are `str` repr equality and hash methods for

`__str__` provides a user-friendly representation; `__repr__` provides a useful developer representation. `__eq__` defines equality. When equality is customized, hashing must remain consistent if the object is to be hashable. Avoid exposing credentials or large payloads in either representation.

A class that defines equality without a suitable hash is commonly unhashable. For unsupported operand types, comparison methods can return `NotImplemented` to allow Python's comparison machinery to try the appropriate alternative. [1]

Q058 What are dataclasses useful for

A dataclass generates routine methods such as initialization and representation from annotated fields. It is useful for value objects and structured internal data. Use `field(default_factory=list)` for an independent list in every instance.

```
from dataclasses import dataclass, field
```

```
@dataclass
class Batch:
    name: str
    records: list[int] = field(default_factory=list)
```

Dataclasses do not automatically enforce type hints at runtime. `frozen=True` restricts assignment through normal mechanisms but does not recursively freeze mutable objects stored in fields.

Q059 What does slots change

`__slots__` can restrict instance attributes and reduce per-instance memory by avoiding a normal instance dictionary in suitable classes. Inheritance, weak references, and classes with an inherited dictionary can change the result.

Use measurement before adding slots for performance. It can complicate extensibility and some tooling. A million small records may justify the optimization; a handful of request handlers probably will not. Do not claim a fixed percentage of memory savings without measuring the actual object layout.

Q060 How do multiple inheritance MRO and super work

Python uses a method resolution order to determine where attributes and methods are found in a hierarchy. `super()` continues lookup after the current class in that order; it does not simply mean “call my direct parent.” Cooperative inheritance requires compatible method signatures and consistent use of `super()`.

Mixins work best when they provide a narrow capability and make few assumptions about state. For complex diamonds, inspect `Class.__mro__` and explain initialization order. If reasoning about the hierarchy dominates the design, composition may be easier to maintain.

Q061 When would you use an abstract base class or a Protocol

An abstract base class can express a nominal interface and prevent instantiation until abstract methods are implemented. A Protocol expresses structural compatibility for static type checking: an object can conform by providing the required members without inheriting from it.

Choose based on the contract. A framework plugin hierarchy may benefit from an ABC. A function accepting any object with a `send()` method may benefit from a Protocol. Neither substitutes for tests of the behavioral rules such as retries, ordering, or thread safety.

Q062 What are metaclasses and when are they justified

A metaclass controls class creation and is itself the class of a class. It can enforce class-level rules or construct specialized frameworks. Most application needs can be handled more simply with decorators, descriptors, factories, or `__init_subclass__`.

A strong senior answer explains the mechanism without proposing it everywhere. Metaclasses affect inheritance and can create conflicts when combining libraries. If the only goal is registering subclasses, first consider whether `__init_subclass__` or explicit registration is sufficient.

6 Typing imports packaging and maintainability

Q063 Does Python enforce type hints at runtime

Ordinary type hints are not automatically enforced when a function is called. Static tools can inspect them before execution, while validation libraries can separately enforce runtime rules. Type annotations document intent, but a caller can still pass an unexpected object unless something checks it.

Distinguish `list[int]` as an annotation from validation of every item in incoming JSON. Trust boundaries need runtime checks. Internal code benefits from static checking because it finds incompatible assumptions before those paths execute.

Q064 How do `Any` object and optional types differ

`Any` largely opts out of static checking for operations involving that value. `object` accepts any object but requires narrowing before type-specific operations. `T | None` means the value may be either `T` or `None`; it does not by itself mean a function argument can be omitted.

```
def display_name(name: str | None) -> str:
    return "Anonymous" if name is None else name
```

The caller still needs to pass `name` unless the signature supplies a default. This distinction also matters when defining validation schemas.

Q065 What are generics `TypedDict` and overloads

Generics preserve relationships between types, such as a function returning the same element type that its input contains. `TypedDict` describes the expected keys and value types of a dictionary for static tools. Overload declarations describe different call signatures while one implementation handles runtime execution.

Do not use an enormous union or `Any` to silence a design problem. Sometimes splitting one ambiguous function into two named functions makes both the implementation and the type contract simpler.

Q066 What is a module and how does importing work

A module is a unit of Python code with its own namespace. Importing normally locates a module, initializes it if needed, and binds a reference. Already imported modules are normally reused through the import cache rather than reexecuted on every import.

Top-level side effects therefore deserve care. Opening connections, launching threads, or running expensive work during import makes tests and worker startup fragile. Keep module import lightweight and perform resource initialization through explicit application startup.

Q067 Why do circular imports happen

Two modules can depend on each other before either finishes initializing. A name expected by one may not yet exist in the other. Fix the dependency direction by moving shared contracts to a lower-level module or separating orchestration from domain logic.

A local import can postpone a dependency and may be a useful tactical fix, but it does not always solve the underlying design problem. Also avoid naming your file after a standard module such as `json.py`, which can shadow the module you intended to import.

Q068 What is the main guard for

if `__name__ == "__main__"`: limits code to execution as the entry-point module. It prevents that code from running merely because another module imports it. It is especially important for multiprocessing patterns that import the main module in child processes.

Keep reusable functions outside the guard and command-line orchestration inside a `main()` function. This makes the code importable and testable. The exact child-process behavior depends on platform and selected multiprocessing start method. [4]

Q069 What problem do virtual environments solve

A virtual environment isolates a project's Python package installation from other projects using the same machine. It does not automatically isolate operating-system libraries, hardware, network permissions, or secrets. Containers address a different layer of reproducibility.

A reproducible project records its interpreter constraints and dependency resolution. A list of direct dependencies alone may not identify every transitive version. Use the project's chosen lock workflow and verify installation in a clean environment.

Q070 What belongs in pyproject.toml

`pyproject.toml` can declare the build system, standardized project metadata, and tool configuration. Build dependencies are needed to build a distribution; runtime dependencies are needed to use it. Keep those roles distinct. [8]

For an application, reproducibility usually requires a compatible lock or deployment constraints workflow in addition to metadata. For a reusable library, overly strict runtime pins can unnecessarily prevent consumers from resolving compatible environments. Explain whether you are packaging an app or a library before recommending a strategy.

Q071 How do wheels source distributions and editable installs differ

A wheel is a built distribution format that can usually be installed without rebuilding the project. A source distribution contains source used to build a distribution. An editable install is useful in development because code changes can be reflected without reinstalling a normal build each time, subject to backend behavior.

Native extensions introduce platform and interpreter compatibility constraints. Test the actual artifact you plan to publish or deploy. A successful import from a source checkout does not prove that package data, dependencies, or entry points were included in the built wheel.

Q072 What makes Python code maintainable

Use meaningful names, small cohesive functions, clear dependency boundaries, stable error contracts, and tests of behavior. Formatting and linting reduce distracting inconsistency, while code review checks reasoning and design. Type checking can catch another class of mistakes.

Do not confuse a short function with a simple design. A one-line expression that hides network calls or mutation can be harder to understand than several explicit steps. Optimize for the next engineer's ability to change the code without breaking an invariant.

7 Complexity and coding interview patterns

Q073 How do you explain time and space complexity

Time complexity describes how work grows with input size, while space complexity describes how memory grows. State what n represents and distinguish auxiliary space from storage required for the output. Average-case and worst-case bounds answer different questions.

A dictionary-based algorithm may be average $O(n)$ while depending on average constant-time hashing. An algorithm over Python integers may also depend on integer size, because arithmetic on arbitrarily large integers is not always constant time. Use the level of detail appropriate to the constraints.

Q074 Which operation costs matter most in Python interviews

For typical built-in collections, list indexing is $O(1)$, list membership is $O(n)$, appending is amortized $O(1)$, and inserting at the front is $O(n)$. Dictionary and set lookups are average $O(1)$. Sorting n comparable items is $O(n \log n)$ in the general case, with existing order sometimes reducing work.

Also account for hidden work: slicing copies list references, building a set consumes the input, and concatenating immutable sequences may copy data. Calling an $O(n)$ operation inside an $O(n)$ loop can create quadratic behavior.

Q075 When should you use two pointers

Two pointers track positions whose movement reduces unnecessary rescanning. They are useful for sorted pair searches, in-place partitioning, palindrome checks, and merging ordered sequences. Explain an invariant, such as all positions outside the current interval already being validated.

The technique is not automatically valid for every array problem. A sorted two-sum approach relies on monotonic order: increasing the left value increases the sum and decreasing the right value reduces it. Without sorting, those movements do not justify discarding candidates.

Q076 When should you use a sliding window

A sliding window maintains information about a contiguous interval while moving its boundaries. Fixed-size windows often add one item and remove one item per step. Variable-size windows adjust boundaries until a validity condition is restored.

Be careful with assumptions. Shrinking a window while its sum is too large works for many positive-number problems, but negative values can break the monotonic reasoning. State why moving a boundary is safe, rather than choosing the pattern merely because the problem mentions a subarray.

Q077 When should you use prefix sums

Prefix sums store cumulative totals so an interval sum can be computed by subtracting two prefixes. They help with many range queries and with counting subarrays using a map of prior prefix totals.

The initial zero prefix is essential because a matching interval can start at index zero. For “number of subarrays summing to target,” increment the frequency of the current prefix after counting earlier matching prefixes; otherwise a zero-length interval may be counted unintentionally.

Q078 What invariant makes binary search correct

Binary search needs an ordered domain or a monotone predicate and a search interval that still contains every possible answer. Each iteration must eliminate part of the interval without discarding a valid answer and must make progress.

Choose one boundary convention and use it consistently. Inclusive bounds and half-open bounds are both valid, but mixing them causes off-by-one errors. For lower-bound problems, the answer may be the insertion position after every existing element.

Q079 When do stacks queues heaps and graphs appear

A stack helps match nested structures and revisit recent state. A queue supports breadth-first traversal. A heap selects the next smallest or highest-priority candidate without sorting all candidates repeatedly. A graph models relationships that are not necessarily a simple sequence or tree.

For breadth-first search, mark a node visited when it is enqueued to avoid adding it repeatedly. For heap entries with equal priority, include a comparable tie-breaker if the payload objects cannot be compared.

Q080 How do you recognize dynamic programming

Dynamic programming is useful when a problem has overlapping subproblems and a solution can be built from smaller states. Define the state, transitions, base cases, and evaluation order. Then decide whether memoized recursion or an iterative table is easier to understand.

Do not start with a table before knowing what each entry means. For a minimum-cost problem, explain whether the state represents reaching an index, leaving an index, or processing a prefix. Different definitions lead to different base cases and return positions.

Q081 How should you test an interview algorithm

Test a normal example, the smallest valid input, duplicates, all-equal values, missing answers, and boundary values. Add negative values or Unicode when allowed. Explain how you know the expected output independently of the code.

For optimized algorithms, compare with a small brute-force solution on many small inputs. This is particularly useful for sliding windows, interval boundaries, and counting problems. A handful of examples can miss an incorrect invariant that randomized comparison reveals quickly.

Q082 What makes an optimization convincing

Show what repeated work is removed and how the complexity changes. Then explain any new memory cost or restriction. Replacing variable names or removing a temporary value is usually not an algorithmic improvement.

For a maximum average over length k , repeatedly summing each slice costs roughly $O(nk)$. Updating one running sum changes the work to $O(n)$. Since k is fixed across candidate windows, compare sums and divide only once at the end.

8 Concurrency parallelism and Python internals

Q083 How do concurrency and parallelism differ

Concurrency means multiple tasks can make progress during overlapping periods. Parallelism means work executes at the same time, typically on multiple cores or devices. A single event-loop thread can handle many waiting network operations concurrently without executing their Python code in parallel.

Choose based on the bottleneck. Waiting on many independent APIs suggests asynchronous I/O or threads. Heavy Python computation often suggests processes or another execution strategy. These are starting hypotheses to validate with measurements, not universal speed guarantees.

Q084 What is the GIL and does every Python runtime have it

In a conventional GIL-enabled CPython build, the global interpreter lock restricts simultaneous execution of Python code by threads within an interpreter. It does not mean that all I/O or native-extension work is serialized, and it does not make application logic race-free.

Optional free-threaded CPython builds were introduced in Python 3.13 and can run Python threads in parallel. Extension compatibility and whether the GIL is actually enabled still matter. Always identify the implementation, build, and workload when answering this question. [2]

Q085 Is Python thread safe because of the GIL

No. Thread safety is a property of operations and invariants, not a blanket property of a language. A check followed by an update can race even if individual container operations preserve internal consistency. Threads may switch between steps, and operations may call user code or release the GIL.

Use synchronization or ownership boundaries for shared state. Do not claim that Java is inherently thread-safe or that Python is inherently unsafe. Both require correct coordination when multiple execution paths share mutable data.

Q086 When should you use threads

Threads are often useful for concurrent blocking I/O and for native libraries that release the GIL. They share memory, so communicating references is convenient but introduces synchronization requirements. Limit their number and size connection pools consistently.

If ten threads all block on a database with only two available connections, adding more threads may increase waiting without increasing throughput. Measure queue time, service time, and resource saturation. Free-threaded builds add another option for CPU work but do not remove data races. [2]

Q087 When should you use processes

Processes provide separate address spaces and can execute CPU-heavy Python work in parallel on multiple cores. Costs include startup, serialization, interprocess communication, and duplicated memory. Keep tasks large enough that useful work outweighs coordination overhead.

Start methods differ. Python 3.14 changed defaults, including using forkserver on supported POSIX platforms; fork is no longer the default anywhere. Avoid assuming child processes inherit all live resources safely. Use importable worker functions and an appropriate main guard. [4]

Q088 What do locks semaphores events and queues solve

A lock protects a critical section. A semaphore limits simultaneous access to a finite resource. An event signals a condition. A queue transfers work or results while helping coordinate producers and consumers.

Choose the primitive that expresses the invariant. A queue can reduce the amount of shared mutable state, while a lock can make a small read-modify-write operation safe. Keep lock scope short and avoid unpredictable network calls while holding a lock.

Q089 What causes deadlocks and how do you prevent them

A deadlock occurs when participants wait indefinitely for resources held by one another. Acquiring multiple locks in inconsistent orders is a common cause. Other causes include waiting for a task that needs a resource you still hold.

Use a consistent acquisition order, reduce nested locking, set appropriate timeouts, and design ownership clearly. Timeouts can limit waiting but do not automatically restore consistency; you still need to know which operations completed before the timeout.

Q090 What does async def actually create

Calling an async def function creates a coroutine object. Its body runs when the coroutine is awaited or scheduled appropriately. Merely calling it does not perform the intended work. await suspends the coroutine when the awaited operation needs to wait and allows other ready work to run.

An async function containing CPU-heavy work or synchronous blocking I/O can still block the event loop. Adding the keyword does not convert a blocking library into a nonblocking one.

Q091 When are gather and TaskGroup useful

Both can coordinate multiple awaitables, but their failure behavior differs. TaskGroup, available from Python 3.11, gives related tasks a structured lifetime and normally cancels remaining siblings after a non-cancellation failure. Failures can be reported through exception groups. gather has different default propagation and sibling-cancellation behavior. [3]

Choose whether tasks belong to one all-or-fail operation or whether partial results are valuable. Do not accidentally leave work running after the request that owned it has failed.

Q092 How should cancellation and timeouts be handled

Use a deadline for the whole operation and bounded timeouts for individual dependencies. Clean up in finally. If catching CanceledError, normally propagate it after cleanup so structured concurrency and timeout handling remain correct. [3]

A timeout tells you that the caller stopped waiting; it does not prove that the remote side did nothing. Payment-like operations therefore require idempotency and status reconciliation. Cancelling an awaiting coroutine also does not reliably stop a synchronous function already running in a thread.

Q093 Why is bounded concurrency important

Scheduling a million tasks can exhaust memory and overload dependencies even if each task is asynchronous. A semaphore limits active work, but creating a million waiting tasks still consumes memory. A bounded queue and fixed worker set can bound both active and waiting work.

Propagate backpressure toward producers. If the system cannot keep up, choose an explicit policy: wait, reject, shed low-priority work, or persist it durably. An unbounded in-memory queue postpones failure until it is larger and harder to diagnose.

Q094 What are context variables useful for

Context variables carry values such as request identifiers through an execution context without passing them through every function. They fit asynchronous context propagation better than assuming thread-local state is sufficient for many tasks sharing one thread.

They are not a shared database or a substitute for explicit business dependencies. Reset values at the correct scope and verify how propagation behaves when work crosses threads, processes, or queue boundaries. A message sent to another process should carry its correlation metadata explicitly.

Q095 How does Python manage memory

CPython primarily uses reference counting along with mechanisms for collecting reference cycles. Other implementations may behave differently. Releasing the last visible reference does not guarantee that process resident memory immediately falls, because allocators and native libraries may retain memory for reuse.

Investigate leaks by identifying what remains reachable: caches, global collections, closures, exception tracebacks, open resources, and native allocations. A rising memory graph is evidence to investigate, not proof that the garbage collector is broken.

Q096 How would you profile a slow Python service

Start with user-visible latency, throughput, errors, CPU, memory, and dependency timings. Distinguish time waiting for a connection from time executing a query. Use tracing for request paths and a profiler for expensive code sections; use memory tools when allocation or retention is the issue.

Reproduce with representative inputs. A microbenchmark of a tight loop does not explain a production request dominated by network waits. Change one likely bottleneck, compare before and after, and check correctness and tail latency as well as the average.

9 Testing debugging and code review

Q097 What are unit integration and end to end tests

A unit test checks a small behavior in isolation. An integration test checks that components interact correctly, such as application code with a real database. An end-to-end test exercises a larger user workflow across boundaries.

Use a mixture based on risk. Hundreds of mocked unit tests can miss a wrong SQL constraint or a serialization mismatch. Conversely, making every test launch the entire system can slow feedback and make failures difficult to localize.

Q098 What makes a good test

A good test has clear setup, a meaningful action, and assertions about observable behavior. It should fail for a real regression, remain understandable, and avoid depending on unrelated implementation details. Test failure paths and invariants, not just happy-path return values.

For an inventory operation, assert that stock never becomes negative and that duplicate requests do not reserve twice. Those checks matter more than whether a private helper was called exactly once when its call count is not part of the contract.

Q099 What are pytest fixtures and their scopes

Fixtures provide reusable test dependencies and setup. Scopes control how long a fixture instance is reused, such as per function, module, or session. A fixture using yield can perform teardown after the test. [9]

Choose broader scopes only when sharing is safe. A session-scoped mutable object can make tests influence one another. A database fixture should define cleanup and transaction behavior explicitly, especially if application code opens its own connections.

Q100 When should you mock and where should you patch

Mock a boundary when real access is slow, nondeterministic, costly, or outside the test's purpose. Patch the name where the code under test looks it up, which may differ from the module where the original object was defined.

Prefer small fakes for stateful behavior when a complicated collection of mocks becomes hard to understand. Keep some integration tests against the real database or protocol implementation so your fake does not silently teach the application an incorrect contract.

Q101 What do parametrized and property based tests add

Parametrized tests run the same behavior against a clear set of cases. Property-based tests generate inputs and check general properties, such as sorting preserving all elements or encoding followed by decoding returning the original supported value.

The property must be meaningful. “The function does not crash” is usually weaker than checking invariants or agreement with a simple reference implementation. When a generated case fails, retain a understandable regression test for the discovered bug.

Q102 How do you test async and concurrent behavior

Use an appropriate async test runner for coroutine tests. Arrange coordination with events, barriers, or controllable fakes instead of arbitrary sleeps. Test cancellation, timeouts, resource cleanup, and maximum in-flight work.

For a race condition, deliberately force the relevant interleaving. Running the same test repeatedly and hoping the race appears is less reliable. Database concurrency tests need separate transactions or connections; two operations inside one test transaction may not reproduce the production race.

Q103 Does high coverage prove quality

No. Coverage shows which code executed, not whether assertions were strong or important inputs were tested. A test can execute every line and still accept the wrong result. Use coverage to identify blind spots, then assess assertions and realistic failure cases.

Mutation testing can expose weak assertions by changing code and checking whether tests notice, but it also has cost. Choose additional testing based on remaining risk, not a desire to maximize a number without improving confidence.

Q104 How should logging differ from printing

Structured logging lets you control levels, destinations, timestamps, and useful fields. A service log should help answer which operation failed, for which request or job, and at what stage. Avoid secrets, raw authentication headers, and unnecessary personal data.

Do not log the same exception at every layer. Decide which boundary owns the final error record. Include a stack trace for unexpected failures while mapping the client response to a safe, stable error contract.

Q105 What do you inspect in a code review

Start with correctness and invariants, then failure handling, security boundaries, data access, resource lifetimes, tests, and readability. Ask whether the change affects callers or deployment order. Check whether retry logic can duplicate side effects and whether a new cache can return another user's data.

Separate required fixes from preferences. Explain the consequence of an issue and suggest a concrete alternative. A review that only discusses variable names while missing a transaction race is not a strong senior review.

Q106 How do you debug a failure that only happens in production

Compare configuration, traffic shape, data volume, dependency versions, timezones, resource limits, and concurrency. Use existing logs and traces to form a hypothesis. Reproduce with sanitized representative data in a controlled environment when possible.

Contain the impact before making a large speculative fix. A rollback, feature flag, or reduced concurrency may restore service while evidence is collected. Avoid exposing sensitive data or changing several unrelated controls at once, because both can make the incident worse.

10 HTTP APIs authentication and web architecture

Q107 What happens during an HTTP request to a Python service

The client resolves the address, establishes the connection and usually TLS, then sends a request. A proxy or load balancer may forward it to an application server. Middleware, routing, authentication, validation, application logic, and data access produce a response.

Not every request creates a new connection because connections may be reused. In a slow request, inspect these stages separately. DNS, pool waiting, TLS, application execution, and response transfer can each contribute latency.

Q108 What are WSGI and ASGI

WSGI is an interface for synchronous Python web applications and servers. ASGI supports asynchronous interactions and protocols such as WebSockets. The interface, server configuration, framework, and libraries together determine actual concurrency behavior.

Deploying a synchronous handler behind an ASGI server does not magically make every operation nonblocking. Likewise, a WSGI application can use workers or threads for concurrency. Explain the whole request execution model instead of comparing only acronyms.

Q109 What makes an API RESTful

A REST-oriented API uses resources, representations, standard HTTP semantics, and stateless request interactions. In ordinary interviews, explain meaningful resource URLs, methods, status codes, caching behavior, and a consistent error model.

Do not treat REST as “JSON over HTTP” only. A GET endpoint that performs a destructive action violates the expectation that retrieval is safe. A POST can be designed with an idempotency key even though the method itself does not promise idempotency.

Q110 How do authentication and authorization differ

Authentication establishes who or what is making the request. Authorization decides whether that principal may perform a specific action on a specific resource. A valid login does not grant access to every object.

Apply authorization after identifying the target object or constrain the query to authorized objects. Checking only that a user is logged in can expose another user's invoice through a changed URL identifier. Test cross-user and cross-tenant requests explicitly.

Q111 How do sessions and JWTs differ

A server-side session commonly uses an opaque identifier whose state is stored server-side. A signed JWT carries claims that a verifier can validate using the appropriate key and rules. JWT signatures protect integrity; they do not inherently hide the payload.

Compare revocation, expiry, key rotation, token size, browser storage, and operational complexity. A token-based system can still require server-side state for revocation or authorization changes. “Stateless” should describe a specific validation path, not imply that the whole application has no state.

Q112 What are CORS CSRF and XSS

CORS controls which browser origins may read certain cross-origin responses; it is not an authentication mechanism. CSRF tricks a browser into sending an unwanted authenticated request using ambient credentials such as cookies. XSS executes attacker-controlled script in a trusted page context.

Use appropriate CSRF defenses for cookie-authenticated state-changing requests. Escape output, avoid unsafe HTML construction, and apply a suitable content security policy to reduce XSS risk. A non-browser client is not stopped simply because a CORS policy rejects a browser origin.

Q113 How should input and output validation be designed

Validate types, ranges, sizes, formats, allowed fields, and relationships between fields at trust boundaries. Separate request schemas from database models and response schemas. Avoid letting users write internal fields merely because those fields exist on a model.

Output schemas also protect data exposure. A user response should not accidentally serialize a password hash, internal risk flag, or access token. Distinguish validation failures from authorization failures and unexpected server errors.

Q114 How do offset and cursor pagination differ

Offset pagination is easy to understand and supports page numbers, but deep offsets can be expensive and concurrent inserts can shift results. Cursor pagination continues from a stable ordering key and often handles large changing datasets better.

Use a deterministic tie-breaker such as (created_at, id). Decide how nulls, sort direction, filters, and deleted records affect continuation. A cursor is a continuation token, not permission to bypass authorization or tenant filtering.

Q115 What are idempotency keys for

An idempotency key lets a client retry an operation while requesting the same logical result. Persist a key scoped to the relevant caller and operation, a request fingerprint, and the operation's state or response. Reject reuse with incompatible input.

A check-then-insert in Python is not enough under concurrency. Use a unique constraint or equivalent atomic mechanism. Also define what happens when the process crashes after an external side effect but before storing the final response.

Q116 How should API errors and versioning work

Use consistent error shapes with a stable code, a safe message, and a request identifier where helpful. Choose status codes based on meaning rather than returning 200 for every outcome. Do not expose raw stack traces to clients.

Version changes around compatibility. Adding an optional field may be compatible for tolerant clients, while renaming a field or changing pagination semantics may not be. Publish deprecation windows and use contract tests with important clients before removing behavior.

11 Django interview questions

Q117 What does Django provide

Django is a web framework with an ORM, routing, templates, forms, authentication facilities, an administrative interface, and other integrated components. It fits applications where those conventions reduce repetitive work and align with the team's needs.

Explain the request path from URL resolution to a view and response. Django's model-template-view vocabulary is related to common MVC ideas, but memorizing a label is less useful than knowing which component owns persistence, presentation, and request handling.

Q118 How should a Django project be organized

A project contains overall configuration and one or more applications. An app should represent a cohesive capability rather than merely a file-type bucket. Keep views focused on request orchestration and put reusable business rules where they can be invoked by HTTP handlers, management commands, and tasks.

Do not create dozens of tiny apps without a reason. Boundaries should make ownership and dependencies clearer. A service layer can help with substantial workflows, but adding one for every trivial CRUD call can become unnecessary indirection.

Q119 What is a lazy QuerySet

A QuerySet describes a database query and often delays execution until its results are needed. Iteration, conversion to a list, and certain other operations can trigger evaluation. Know whether the operation issues SQL and whether a result cache is reused.

When optimizing, inspect the actual query count and query plan. Avoid assuming that a visually short ORM expression performs one cheap query. Templates, serializers, and related-field access can trigger additional database work. [10]

Q120 What is the N plus one query problem

Fetching n objects and then separately loading a relation for each can create one initial query plus n additional queries. Django's `select_related` uses joins for suitable single-valued relationships. `prefetch_related` performs additional queries and combines results in Python, supporting cases such as collections. [10]

Choose based on relationship shape and result size. Eagerly loading every relation can overfetch. A useful test asserts a query budget for an endpoint with several records so the test can detect accidental per-record queries.

Q121 What are migrations and why can they be risky

Migrations version database schema changes and sometimes data transformations. A migration that works instantly on a local table may lock or rewrite a large production table. Application versions may overlap during rollout, so old and new code must temporarily work with the intermediate schema.

Use expand-and-contract changes for incompatible transitions: add new structures, deploy compatible code, backfill in controlled batches, switch reads, then remove old structures later. Understand the target database's DDL behavior and test on representative scale.

Q122 How do you protect a Django business transaction

Use a database transaction for changes that must commit together and database constraints for invariants. Row locking or conditional updates may be necessary when requests compete for the same state. `transaction.atomic()` gives a transaction scope, but it does not by itself serialize every business decision. [22]

For decrementing inventory, avoid reading stock into Python and blindly saving a reduced value. Use a locked row or an atomic conditional update that succeeds only if enough stock remains. Handle the no-update case as a business conflict.

Q123 When are F expressions useful

An `F` expression lets the database operate on a field value rather than using a possibly stale value read into Python. It is useful for increments and field-to-field comparisons. Combine it with a condition when enforcing an invariant such as nonnegative stock.

Framework fragment:

```
from django.db.models import F

changed = Product.objects.filter(
    pk=product_id, stock__gte=quantity
).update(stock=F("stock") - quantity)
if changed != 1:
    raise InsufficientInventory(product_id)
```

This fragment assumes a positive validated quantity and application-defined model and exception. A complete purchase may require a surrounding transaction and an idempotency record.

Q124 When should you use signals

Signals let loosely connected code respond to events, but implicit execution can make business workflows hard to trace. Use them for limited cross-cutting behavior when decoupling is intentional. Prefer explicit orchestration for steps that must happen in a known order or share a transaction.

Do not assume every bulk operation follows the same model-hook behavior as calling `save()` on each instance. Know which code paths trigger your logic. For reliable external delivery, a transactional outbox is stronger than an in-memory callback alone.

Q125 How should caching be used in Django

Cache data whose freshness requirements are explicit. Include every relevant dimension in the key, such as tenant, user permissions, language, filters, and representation version. Define expiry and invalidation behavior before enabling a shared cache.

Caching an authenticated response under a URL-only key can leak data across users. Also avoid long-lived caching of stale authorization decisions without a revocation strategy. Measure hit rate, miss latency, payload size, and backend load.

Q126 What should you know about Django async support

For the Django 5.2 baseline, async views work with an ASGI request stack and parts of the ORM have async interfaces. Transactional ORM work still needs a synchronous function called through an appropriate adapter; synchronous middleware can also affect the execution model. [11]

Do not disable async-safety checks to force blocking ORM calls into async views. Choose a consistent execution path and verify library compatibility. A mostly synchronous application can be a sound choice when its workload and operating model do not require an async stack.

Q127 How do you secure Django configuration

Treat secret keys and database credentials as environment-specific secrets. Configure trusted hosts, HTTPS-related settings, cookies, CSRF behavior, and proxy trust deliberately. Turn off development debugging in production and avoid serving the application through a development server.

Framework defaults are a starting point, not the entire security model. Application-level object authorization, upload handling, dependency maintenance, and administrative access remain your responsibility. Review the exact deployment topology before trusting forwarded host or scheme headers.

Q128 How would you test and optimize a slow Django endpoint

Capture a representative request trace, count SQL statements, inspect expensive plans, and measure serialization and template rendering. Check relation loading, indexes, result size, and accidental evaluation. Add a targeted regression test for the discovered behavior.

Do not immediately add a cache or more web workers. An unindexed query or N plus one pattern can become more damaging under increased concurrency. Re-measure the endpoint after the smallest fix that addresses the demonstrated bottleneck.

12 Django REST Framework interview questions

Q129 What does a serializer do

A serializer converts between representations and validated application data according to a declared schema. In DRF it can also participate in creating or updating model instances. It is not simply a call to JSON encoding.

Separate read-only fields, writable fields, and server-controlled ownership. Cross-field validation belongs where related input can be considered together. Complex multi-object transactions may be clearer in an explicit service called by the serializer or view rather than hidden inside nested serialization.

Q130 How do APIView generic views and ViewSets differ

An APIView gives explicit control over HTTP handling. Generic views provide common CRUD behaviors. ViewSets group related actions and can work with routers to generate routes. Choose the abstraction that makes the endpoint's behavior clearest.

Do not force every workflow into a model CRUD shape. An operation such as approving a claim may have a richer domain contract and idempotency rules. It can still use framework authentication, permissions, serializers, and responses without pretending it is only a generic row update.

Q131 Where should object permissions and queryset filtering happen

Object permissions determine access to an individual object, while query filtering restricts which objects can appear in a result set. DRF does not automatically apply object-level permission checks to every row of a list response, and creation needs its own ownership checks. [12]

Scope the queryset by the authenticated tenant or user, then apply action-specific rules. Test list, detail, update, delete, and create separately. A correct detail endpoint does not prove that a list endpoint or custom action is safe.

Q132 How do authentication permissions and throttling relate

Authentication identifies the principal. Permissions decide whether an action is allowed. Throttling limits request frequency according to a policy. These controls solve different problems and may all be needed for one endpoint.

Application throttling is not a complete defense against large-scale traffic abuse. Put capacity protection at the appropriate gateway or infrastructure layer as well. Decide whether limits are per user, token, tenant, IP, or operation, and how shared state is coordinated across workers.

Q133 Why do nested serializers often cause performance problems

A nested serializer may access related objects for every row, producing repeated queries or excessive data loading. Inspect the relation access pattern and prepare the queryset accordingly. Also limit nested depth and collection size.

Avoid putting database queries in every computed field without considering list endpoints. A helper that looks harmless for one object can issue hundreds of queries in a paginated collection. Test with multiple objects and record query counts.

Q134 How would you design a DRF bulk update endpoint

Validate batch size, permissions for every target, duplicate identifiers, and whether the operation is all-or-nothing. Decide how individual failures are represented when partial success is allowed. Use transactions consistent with that decision and avoid relying on hooks that bulk operations skip.

For large batches, accept a durable job and provide status rather than keeping an HTTP request open indefinitely. Make retries safe and record a stable operation identifier so the client can distinguish a repeated request from a new batch.

13 Flask interview questions

Q135 What does microframework mean for Flask

Flask keeps its core relatively small and lets applications choose extensions and supporting libraries. It can support substantial applications, but the team must make more explicit choices about database access, validation, project structure, and other facilities.

Microframework does not mean microservice, low traffic, or a one-file production application. Assess whether the flexibility helps your team or creates repeated decisions that an integrated framework would have handled well.

Q136 Why use an application factory and Blueprints

An application factory constructs and configures an application explicitly. It supports different test and deployment configurations and avoids initializing all resources as import-time side effects. Blueprints group related routes and registration behavior.

Initialize extensions in a way that allows them to bind to the created application. Keep business services separate enough that a test can exercise them without fabricating a request context. Do not hide every dependency behind a global variable merely because the framework permits convenient access.

Q137 What are request and application contexts

Contexts make objects such as the current request and application available during the appropriate lifecycle. Their convenient proxies do not mean there is one ordinary global request shared by every user. Access outside the relevant context can fail.

Do not send a live request proxy to a background worker. Extract the small validated values it needs, such as an operation ID, tenant ID, and correlation ID. The worker should establish its own resources and authorization assumptions explicitly.

Q138 How do Flask sessions behave by default

Flask's default session mechanism stores signed data in a client cookie. Signing detects tampering but does not encrypt the contents. [17] Keep secret material out of the session payload and understand cookie size and expiry constraints.

If immediate revocation or large server-controlled state is required, evaluate server-side session storage. Authentication and authorization still need a deliberate design. A signed cookie containing a user ID does not by itself prove that every requested object belongs to that user.

Q139 Does async Flask turn it into an async first server

No. Under the usual WSGI model, a request still occupies a worker even when an async view performs concurrent I/O internally. Async support does not inherently increase the number of requests one worker can handle. Background tasks created in a short-lived view loop are also not a durable job system. [13]

Choose the deployment model based on workload. For predominantly async applications with long-lived connections, consider an ASGI-oriented architecture rather than assuming an async keyword changes the underlying server contract.

Q140 What are common Flask production mistakes

Using the development server, enabling debug mode, trusting arbitrary proxy headers, keeping mutable process-local state as authoritative data, and failing to close database resources are common mistakes. Multiple workers do not share a normal Python dictionary.

Create a repeatable configuration, a production server setup, explicit lifecycle management, and structured errors. Test with the same number and type of workers that matter to correctness; a single local process can hide coordination problems.

14 FastAPI Pydantic and async APIs

Q141 What does FastAPI provide

FastAPI combines an ASGI-oriented API framework with type-driven request declarations, validation integration, dependency injection, and API schema generation. These facilities can reduce repetitive endpoint code, but business rules and authorization remain application responsibilities.

Generated API documentation is useful only when the declared contract matches behavior. Specify response models, error responses, bounds, and descriptions where appropriate. A schema cannot describe every operational guarantee, so document retry and idempotency behavior separately.

Q142 When should a FastAPI endpoint use def or async def

Use `async def` with awaitable nonblocking operations. FastAPI runs ordinary `def` path functions in an external thread pool, which can suit blocking I/O. A synchronous helper called directly inside an `async` endpoint is not automatically moved to that pool. [14]

For CPU-heavy processing, evaluate a process worker or separate compute service. Threads and `async` both need capacity limits. “FastAPI is `async`” is not enough to choose an endpoint execution model.

Q143 How does dependency injection help

Dependencies declare reusable inputs or setup, such as obtaining an authenticated user, opening a session, or checking an access rule. They can depend on other dependencies and can be overridden during tests. [15]

Keep dependency responsibilities clear. A dependency that silently performs writes or calls several unrelated services makes endpoint behavior surprising. Separate authentication, authorization, and business changes so their failure and transaction boundaries remain visible.

Q144 Where should shared clients be initialized

Create suitable long-lived clients during application lifespan and close them on shutdown. Reuse connection pools when the client's concurrency contract allows it. Request-scoped database sessions should still have a separate controlled lifetime.

Remember that each process has its own memory and startup lifecycle. Four workers may create four pools or load four copies of a model. Count this multiplication when estimating connections, memory, and startup time.

Q145 What changed in Pydantic v2 that interviewers may ask

Pydantic v2 uses APIs such as `model_validate` and `model_dump`; older names may remain as deprecated compatibility paths. A nullable field is not automatically optional to omit: an explicit default determines whether absence is allowed. ORM-style attribute reading is configured through `from_attributes`. [16]

State the major version in code discussions. Do not mix v1 configuration and validator examples into a v2 project without checking migration guidance. Validation behavior is part of your API contract, so upgrades need representative payload tests.

Q146 How do strict validation and coercion differ

Coercion converts compatible input forms, such as numeric text into a number when allowed. Strict validation rejects forms outside the declared requirements. Pick the behavior intentionally, especially for identifiers, booleans, and values whose textual form matters.

Coercion can improve usability at a boundary, but it can also hide bad upstream data. Model validation is not business authorization: a syntactically valid amount or account identifier still needs contextual checks.

Q147 How would you prevent sensitive fields in responses

Declare a response schema containing only permitted public fields and map internal objects into it. Keep input and output models separate when their fields differ. Test that secrets are absent rather than assuming the default serializer will always remain safe after a model changes.

If different roles see different fields, make that policy explicit and include role or authorization scope in any cache key. Do not rely on the frontend to hide fields that the API has already transmitted.

Q148 When are background tasks insufficient

In-process background work is unsuitable when a task must survive process crashes, deployments, or host loss. Use a durable queue and worker for important or long-running work. Store job state and return an operation identifier. [21]

Also decide how the job is committed relative to the database change that created it. Committing a row and then publishing a message leaves a failure gap. An outbox closes that local gap, while consumers still need idempotency.

Q149 How do you test a FastAPI application

Test request parsing, dependency overrides, authentication failures, response schemas, and domain outcomes. Ensure the chosen test setup runs application lifespan when startup resources are needed. Use integration tests for database and external protocol assumptions.

For async clients, verify that application resources belong to a compatible event-loop lifecycle. Test shutdown and cleanup as well as request success. A test client returning 200 does not demonstrate safe concurrent access to one shared session.

Q150 How do WebSockets and server sent events differ

WebSockets provide bidirectional communication over a persistent connection. Server-sent events provide a server-to-client event stream over HTTP and can fit progress updates or token streaming. Choose based on directionality, infrastructure support, reconnection behavior, and message semantics.

Persistent connections need backpressure, heartbeat or liveness decisions, limits, and cleanup.

Reconnection can duplicate events, so include identifiers and define replay behavior when delivery continuity matters. Authentication may need re-evaluation as a long-lived connection ages.

15 SQL databases and SQLAlchemy

Q151 Why must a Python backend engineer know SQL

An ORM constructs database operations, but the database still executes queries, chooses plans, acquires locks, and enforces constraints. Python code that hides SQL can still cause slow joins, excessive queries, or lost updates.

Understand joins, grouping, indexes, transactions, null semantics, and execution plans. Be able to explain which data is filtered in the database and which is materialized in Python. Pulling a million rows into a list to keep ten of them is usually an architectural problem before it is a Python optimization problem.

Q152 How do inner and left joins differ

An inner join returns matching combinations. A left join also retains left-side rows without matches, supplying nulls for the missing right side. A filter on the right table placed in WHERE can accidentally remove those unmatched rows and change the intended behavior.

Clarify whether a relationship is one-to-one, one-to-many, or many-to-many. Joins can multiply rows, so a sum or count after a join may be inflated. Validate cardinality before adding DISTINCT as an unexplained fix.

Q153 How does NULL differ from an ordinary value

SQL null represents missing or unknown information and follows three-valued logic. Use IS NULL, not equality to null. Aggregate and comparison behavior must be understood; for example, counting all rows differs from counting non-null values in one column.

Do not assume Python's None comparison behavior exactly mirrors SQL. Also be careful with NOT IN when the subquery can contain nulls. A NOT EXISTS formulation may express the intended relationship more clearly.

Q154 How do indexes improve queries and what do they cost

An index gives the database another access path that can reduce scanned data or sorting for suitable queries. It consumes storage and adds maintenance work during writes. An index on every column is rarely a good strategy.

For a multicolumn B-tree index, column order and predicates matter, although database features and planners can support additional access strategies. Inspect the actual plan instead of claiming that a later column can never be used. [19] Choose indexes from observed query shapes and data distribution.

Q155 What are ACID and isolation levels

Atomicity groups changes, consistency concerns valid state, isolation controls interaction between concurrent transactions, and durability concerns committed data surviving the relevant failures. Database consistency does not mean every business invariant is automatically enforced without constraints or correct transactions.

Isolation levels permit different anomalies. In PostgreSQL, Read Committed gives statement-level snapshots; Serializable can reject conflicting executions, requiring a retry of the transaction. The retry must repeat the whole logical transaction safely. [18]

Q156 What are pessimistic and optimistic concurrency controls

Pessimistic control acquires locks before competing changes. Optimistic control checks that a version or expected value has not changed before committing an update. Both need conflict handling and a strategy for retries or rejection.

An optimistic update can use `WHERE id = ? AND version = ?`, incrementing the version and checking the affected-row count. A zero-row update indicates a conflict or missing record. Do not silently overwrite another request's newer changes.

Q157 What is a SQLAlchemy Session

A Session manages a unit of work and an identity map for ORM objects. An Engine manages database connectivity and pooling. A Session is mutable transaction-related state, not a global shared application cache. Use a Session per thread or an AsyncSession per concurrent task. [5]

Keep ownership and transaction boundaries explicit. Passing one session through the functions of one operation can be appropriate; sharing it across unrelated concurrent requests is not. Close or release it reliably when the operation ends.

Q158 How do flush commit and rollback differ

Flush sends pending ORM changes to the database within the current transaction. Commit completes the transaction; rollback discards its uncommitted work. A flush does not mean a change is durably committed. SQLAlchemy can flush automatically before certain operations. [5]

Design transaction boundaries around business operations, not every helper call. If one helper commits halfway through a larger workflow, the caller may be unable to roll back the whole change when a later step fails.

Q159 What problems occur with lazy loading in async ORM code

Accessing an unloaded relationship can require database I/O. In async code, that I/O must occur through supported awaitable paths; implicit access can fail or produce unexpected work. Plan eager loading or explicitly load required values using the library's async facilities. [23]

Do not return partially loaded ORM objects and hope response serialization will safely fetch everything. Build the response while the data-access lifetime and loading strategy are clear. This also reduces accidental N plus one queries.

Q160 How should database pool sizes be chosen

Count all processes and replicas. Ten replicas with four processes and a maximum of ten connections per process can potentially request 400 connections, before other services and administration. The database's safe capacity may be much lower.

Use bounded pools and timeouts, and measure pool wait time separately from query execution. Increasing a pool can worsen database contention. Avoid holding a transaction open while waiting on a slow external API unless the consistency requirement truly demands it and the risk is controlled.

Q161 What is an upsert and when is it not enough

An upsert inserts or updates according to a uniqueness conflict. It can support idempotent ingestion when the key and update rule are correct. It does not automatically resolve out-of-order updates, deletes, or incompatible versions.

For source replication, compare a source version or timestamp and define tie-breaking. If two different records have the same modified timestamp, a timestamp-only rule may lose information. Preserve tombstones or another deletion signal when removals must propagate.

Q162 How do you test database correctness

Test unique constraints, foreign keys, transaction rollback, concurrent updates, duplicate requests, and migration compatibility. Use the production database engine for behavior that depends on its SQL dialect or locking semantics.

An in-memory substitute can be useful for quick unit tests, but it cannot prove PostgreSQL isolation behavior. Keep a small set of focused integration tests that exercise the actual failure mechanisms your design relies on.

16 Caching queues and distributed workflows

Q163 What is cache aside and what can go wrong

With cache-aside, read the cache first; on a miss, read the source and populate the cache. Writes update the source and invalidate or refresh the cache according to a policy. Failure windows can leave stale data.

Choose a tolerable freshness bound and define how the system behaves if the cache fails. Cache keys should include authorization scope and schema version where relevant. An in-process cache is separate in each worker and disappears on restart.

Q164 What are cache stampedes and hot keys

A stampede happens when many requests regenerate the same expired value at once. A hot key receives disproportionate traffic and can concentrate load on one cache node or one regeneration path.

Possible controls include request coalescing, staggered expiry, limited regeneration, and serving a bounded stale value while refreshing when permitted. A lock used for regeneration needs a timeout and failure behavior. Do not let every request wait indefinitely for a worker that crashed.

Q165 What is a task queue and what does Celery add

A queue separates accepting work from executing it. Celery adds task dispatch and worker facilities, with a broker for message transport and an optional result backend. Broker acceptance, task execution, and business completion are separate states.

Choose acknowledgement behavior deliberately. Late acknowledgement can be useful for idempotent tasks, but worker termination and broker configuration affect redelivery details. Do not promise unconditional exactly-once execution based on one Celery setting. [20]

Q166 Why must consumers handle duplicate messages

In many reliable delivery systems, a worker can finish a side effect and crash before its acknowledgement is recorded. The message can then be delivered again. Deduplicate using a stable event or operation ID and persist the outcome with the business change when possible.

Checking an in-memory set is insufficient across restarts and replicas. A unique database record can make a local operation idempotent, but an external side effect still needs a remote idempotency key or a reconciliation process.

Q167 How should retries be designed

Retry only failures that may be transient and operations that are safe to repeat. Use a bounded attempt count, an overall deadline, exponential backoff, and jitter when appropriate. Honor meaningful server retry instructions and avoid retrying a validation failure.

Coordinate retries across layers. If a client retries three times, a gateway retries three times, and a worker retries three times, one logical action can create many attempts. Assign a clear retry owner and track attempts by operation ID.

Q168 What are a dead letter queue and a poison message

A poison message repeatedly fails because of its content or an unrecoverable condition. A dead-letter destination retains failed messages after the defined retry policy so they can be inspected, repaired, or replayed.

Record enough context to diagnose the failure without storing unnecessary sensitive payloads. Replaying should be an explicit operation with idempotency and auditability. Moving everything to a dead-letter queue is not success; alert on age, growth, and business impact.

Q169 What is the transactional outbox pattern

Write the business change and an outgoing event record in the same database transaction. A separate publisher reads pending events and sends them to the broker. This avoids a local gap where the database commits but the application crashes before publishing.

Publishing can still happen more than once if acknowledgement of publication is lost. Consumers must remain idempotent. Also define ordering, retention, publisher coordination, and how old unpublished records are detected.

Q170 What is a saga

A saga coordinates a multi-step workflow across services using local transactions and compensating actions. A compensation is a business operation that attempts to undo or offset an earlier effect; it is not necessarily a perfect rollback.

For a reservation, charging and later refunding are different externally visible events. Model pending, completed, failed, and compensating states explicitly. Decide who orchestrates transitions and what happens when the compensation itself fails.

Q171 What do circuit breakers and bulkheads solve

A circuit breaker stops repeatedly calling a dependency that is failing and later probes for recovery. A bulkhead limits how much of one shared resource a workload can consume, preventing one failing dependency or tenant from exhausting everything.

Neither replaces timeouts or capacity planning. Define fallback behavior honestly: returning stale data may be acceptable for a catalog, while pretending a reservation succeeded is not. Observe state changes so a breaker does not hide a persistent outage.

Q172 What does backpressure look like across a system

Backpressure means downstream capacity limits influence upstream production. A bounded worker queue, admission control, and a retryable rejection can prevent unlimited accumulation. At each boundary, define whether the producer waits, persists work, or receives a failure.

Queue depth alone is insufficient: a thousand one-second jobs differ from a thousand one-hour jobs. Track oldest-job age, processing rate, task duration, and deadlines. Autoscaling should consider these signals and the limits of downstream services.

17 Security reliability and deployment

Q173 What Python security mistakes should an interviewer expect you to catch

Watch for SQL built through string interpolation, shell commands assembled from untrusted text, untrusted pickle loading, unsafe template construction, path traversal, and missing object authorization. Also consider unbounded parsing and file-processing resources.

Use parameterized queries, argument lists for subprocesses where suitable, constrained file paths, and explicit permission checks. Avoid treating validation as a universal defense; a valid URL can still point to an internal service that the application should never fetch.

Q174 What is SSRF and how do you reduce its risk

Server-side request forgery makes a server issue requests to unintended destinations. A feature that fetches a user-supplied URL can expose internal services or cloud metadata endpoints if destinations are not controlled.

Prefer allowlisted destinations when possible. Validate schemes, resolved destinations, redirects, and network egress behavior; account for DNS changes. Apply response-size and timeout limits. URL syntax validation alone does not establish that the destination is permitted.

Q175 How should passwords and secrets be handled

Use a maintained password-hashing implementation with an appropriate adaptive scheme and framework guidance. Do not store passwords in plaintext or use a fast general hash as the password storage design. Keep service secrets out of code, logs, images, and public configuration.

Use least-privilege access and a rotation plan. Short-lived workload credentials can reduce the burden of distributing long-lived keys. A secret-management system helps with storage and delivery, but the application must still avoid leaking values after retrieval.

Q176 What should a production container contain

Build the application and dependencies reproducibly, use an appropriate runtime image, run with minimal required privileges, and keep secrets outside the image. Start the intended application process directly or through a signal-forwarding entry point.

Do not assume a container has unlimited CPU or memory. Worker counts should respect the actual resource allocation. A small image can improve distribution and reduce unnecessary components, but correctness and maintainability matter more than minimizing bytes at any cost.

Q177 How do readiness liveness and startup checks differ

Readiness asks whether the instance should receive traffic. Liveness asks whether it should be restarted. A startup check allows initialization to finish before other checks become decisive. Design each around the action it triggers.

If a liveness check fails whenever a shared database is unavailable, every healthy application process may restart during a database outage. That can add load and slow recovery. A check should be inexpensive and should not create a new failure loop.

Q178 What is graceful shutdown

An instance stops accepting new work, finishes or safely hands off in-flight work within a deadline, releases resources, and exits. Long-running jobs need a checkpoint, retry, or lease strategy rather than assuming shutdown always has enough time.

Coordinate load-balancer draining, server timeouts, worker behavior, and orchestration termination windows. If a job is interrupted after a side effect, recovery depends on persisted state and idempotency, not simply catching an exception during shutdown.

Q179 What should a CI pipeline verify

Check formatting and linting, static types where used, focused unit tests, important integration tests, packaging, and deployment compatibility. Add dependency and image scanning appropriate to the organization. Build the artifact once and promote that artifact rather than rebuilding differently for each environment.

Keep feedback fast by separating cheap checks from slower suites. A green pipeline proves only the checks it ran. Include migration and configuration validation when those are the material risks of the change.

Q180 How do rolling canary and blue green deployments differ

A rolling deployment replaces instances gradually. A canary exposes a limited share of traffic to a new version and compares behavior. Blue-green maintains separate environments and switches traffic between them. Each has cost and operational trade-offs.

All need compatible database and message schemas during overlap. A rollback of application code may not reverse a destructive migration. Define which metric or business signal stops a rollout and how traffic and data remain recoverable.

Q181 What are SLIs SLOs and error budgets

An SLI is a measured indicator such as the fraction of successful requests within a latency threshold. An SLO is the desired level over a defined window. An error budget represents the allowed shortfall and can guide rollout and reliability priorities.

Define the denominator and exclusions honestly. An availability metric that ignores all failed requests tells a misleading story. For asynchronous systems, user-visible completion time may be more useful than the speed of accepting a job.

Q182 Which production metrics are most useful

Track request rate, error rate, latency percentiles, saturation, database pool waits, queue age, task failures, and resource usage. Add business metrics such as completed imports or successful reservations. Avoid high-cardinality labels such as every request ID in a metrics series.

Use logs for specific events, metrics for trends and alerting, and traces for causal request paths. Sampling and retention should preserve useful evidence without storing every payload indefinitely.

Q183 How would you handle an incident

Establish impact and ownership, stabilize the service, and communicate concrete status. Use evidence to choose mitigation, such as rollback or reduced load. Keep a timeline of actions and outcomes so later investigators can distinguish correlation from cause.

After recovery, examine contributing conditions and add a small number of effective improvements. A useful review explains why the system allowed the failure and how detection or containment can improve. Blaming the person who triggered a fragile path does not make the system safer.

Q184 How do you plan disaster recovery

Agree on acceptable data loss and recovery time, then design backups, replication, and restore procedures to match. A backup that has never been restored is an unverified assumption. Protect backups and test recovery with representative data and dependencies.

Know what replication does not protect against: an accidental deletion or corruption may replicate quickly. Recovery also includes credentials, configuration, queue state, and application compatibility with the restored database.

18 Data engineering numerical Python and AI applications

Q185 How do Python lists and NumPy arrays differ

A list stores references to Python objects and supports mixed types. A NumPy array uses a typed multidimensional data representation that supports efficient array operations. Many numerical operations execute in optimized native code.

Vectorization can reduce Python loop overhead, but large intermediate arrays can increase memory use. Measure the complete operation, including conversion and copying. A small input may not benefit enough to justify a more complicated array implementation.

Q186 What is NumPy broadcasting

Broadcasting combines compatible array shapes by comparing trailing dimensions; dimensions are compatible when equal or when one is 1. It can avoid explicitly copying an operand, but the resulting computation can still create a large output. [6]

For a matrix of samples and a vector of feature offsets, broadcasting can subtract the offset from every row. Check axis meaning carefully. A shape that is technically compatible can still compute the wrong business result if rows and columns were misunderstood.

Q187 What pandas pitfalls matter in interviews

Know missing values, dtype choices, merge cardinality, grouping, indexing, and memory cost. Prefer explicit column operations to row-wise Python loops when the operation fits. Validate merge keys so many-to-many joins do not unexpectedly multiply records.

For pandas 3.x, Copy-on-Write is the default behavior and chained assignment is not a reliable way to update the original object. Use a single explicit assignment such as `.loc[...] = ...` and test against the project's pinned version. [7]

Q188 How do you process data larger than memory

Read in bounded batches, push suitable filters and aggregates into a database or query engine, use columnar storage where appropriate, and avoid converting everything into one DataFrame. Consider an out-of-core or distributed engine when a single process is insufficient.

Batching alone does not solve operations requiring global state, such as an exact global sort. Those require an external merge, partitioning, or another deliberate algorithm. Explain where intermediate state lives and how a failed batch is retried.

Q189 What makes an ETL pipeline idempotent

A repeated logical run produces the intended target state without duplicate effects. Use stable source keys, version-aware upserts, durable checkpoints, and deterministic transformations. Record which input object or source interval was processed.

Do not advance the checkpoint merely because extraction succeeded if loading later failed. Tie progress to the committed target outcome. Define how to handle source deletions, late updates, rejected records, and schema changes.

Q190 Why are timestamp watermarks tricky

Equal timestamps, clock differences, delayed visibility, and pagination during ongoing source updates can skip or duplicate records. Use a stable ordering and a composite watermark when supported, or deliberately overlap intervals and deduplicate.

Capture an extraction upper bound when the source supports consistent filtering, and do not claim snapshot consistency if it does not. Reconciliation should compare more than row counts when updates and duplicates can preserve the count while changing the data.

Q191 How should a scheduler orchestrate Python data jobs

Represent dependencies and retry policy explicitly. Jobs should be independently rerunnable and should record their input window and output version. Separate schedule time from the data interval being processed so a delayed run does not silently process the wrong date.

Use retries for transient failures and backfills for historical intervals, with concurrency limits to protect shared systems. Keep task payloads small and place large data in durable storage. A scheduler coordinates work; it should not become the only copy of business data.

Q192 What changes when Python serves an ML model

Model loading, preprocessing, inference, and postprocessing form one versioned contract. Inputs must match the training-time feature meaning, not merely have the right number of fields. Measure latency, batch behavior, memory, and hardware utilization.

Avoid loading a large model for every request. Also avoid multiplying model memory blindly across web workers. Use an appropriate inference process or service and define behavior when a model version is unavailable or an input is outside the supported domain.

Q193 What is RAG and how would you explain its pipeline

Retrieval-augmented generation retrieves relevant external material and supplies it as context to a language model. A typical pipeline ingests documents, extracts text, chunks it, creates embeddings or search indexes, retrieves candidates, optionally reranks them, and generates an answer with references.

Retrieval quality and answer quality are separate. A fluent answer can still be unsupported. Evaluate whether the right passages were retrieved, whether the answer is faithful to them, and whether the system declines to invent an answer when evidence is missing.

Q194 What are the most important RAG engineering risks

Enforce access controls during retrieval, propagate document deletion, track versions, and prevent one tenant's content from reaching another. Retrieved text is untrusted input, so it must not gain authority to change tool permissions or system behavior.

Version the embedding model and chunking strategy together with the index. Querying vectors from incompatible embedding spaces can produce meaningless results. Log retrieval identifiers and evaluation outcomes without unnecessarily retaining sensitive prompts and documents.

Q195 How should LLM retries streaming and structured outputs work

Bound deadlines, request sizes, output lengths, and cost. Validate structured output against an explicit schema and handle validation failure rather than trusting a prompt instruction. If streaming has already sent partial output, a transparent retry may produce duplicate or contradictory content.

Separate retrievable generation from side effects. A model suggesting a tool call does not mean the action is authorized or idempotent. Validate arguments, enforce permissions in deterministic code, and record operation identity before executing important changes.

Q196 What makes an agent workflow production ready

Define allowed tools, state transitions, stopping conditions, budgets, and approval requirements for consequential actions. Persist state when a workflow must survive interruptions. Keep tool outputs distinct from trusted instructions and validate every external effect.

Use a framework only when its abstractions help with these requirements. The engineering questions remain the same whether the workflow uses LangGraph, another orchestration library, or explicit Python: how does it recover, avoid duplicate actions, enforce permissions, and demonstrate useful outcomes?

19 Senior engineering and architectural judgment

Q197 What distinguishes a senior Python answer from a junior answer

A junior answer may correctly describe a language feature. A senior answer also identifies the operating assumptions, failure modes, compatibility constraints, and cost of maintaining the solution. It connects the feature to a measurable user or business requirement.

For caching, a senior answer covers invalidation and isolation. For async, it covers blocking dependencies and bounded concurrency. For a queue, it covers duplicate delivery and recovery. Depth means understanding consequences, not merely naming more tools.

Q198 When should you choose Django Flask or FastAPI

Choose according to application needs and team familiarity. Django can reduce work when its integrated facilities fit. Flask gives flexibility for a smaller chosen stack. FastAPI offers an ASGI-oriented API workflow with type-driven validation and schema tooling.

There is no universally fastest or best choice. Compare the actual workload, library compatibility, administration needs, operational model, and development cost. For a database-bound CRUD service, query design may dominate differences between frameworks.

Q199 When is a modular monolith better than microservices

A modular monolith keeps deployment and transactions simpler while preserving internal boundaries. It is often effective when one team owns the domain and independent scaling or deployment is not yet required. Microservices add network, versioning, observability, and distributed consistency costs.

Extract a service when there is evidence: a distinct ownership boundary, independent release needs, different resource profile, or meaningful isolation requirement. A folder boundary can be strengthened before introducing a network boundary.

Q200 How do you make a build versus buy decision

Compare required capabilities, integration cost, reliability, security, migration options, operational ownership, and total cost over time. Include the cost of failure and the difficulty of leaving the vendor. A managed tool can reduce work but still requires configuration and monitoring.

State which requirement is decisive. If no unusual requirement exists and the team is small, an established managed service may be preferable. If a proprietary workflow creates substantial constraints, a focused custom component may be justified.

Q201 How would you modernize a legacy Python application

First map critical behaviors, dependencies, deployment assumptions, and current failures. Add characterization tests around important boundaries, then make small reversible changes. Upgrade supported runtime and library versions in controlled steps with compatibility checks.

Do not rewrite everything merely because the code is old. A rewrite can reproduce hidden bugs while losing undocumented business behavior. Use incremental replacement when it preserves service continuity and lets the team validate value sooner.

Q202 How should a senior engineer discuss performance targets

Translate a target into workload, latency percentiles, throughput, resource limits, and cost. “Handle a million users” is incomplete without active-user rate and request patterns. Distinguish average traffic from peaks and individual request latency from total job completion time.

Present estimates as assumptions, then design a load test to validate them. If the bottleneck is uncertain, say what measurement will resolve it. Avoid guaranteeing throughput from framework marketing or a generic worker-count formula.

Q203 What should an architecture decision record contain

State the problem, constraints, considered options, decision, consequences, and triggers for reconsideration. Keep it short enough that another engineer will read it. Record rejected options fairly rather than inventing weak alternatives to justify the choice.

The purpose is future understanding. A decision that was reasonable for a small team may need revision after growth. Recording its assumptions makes that revision easier and avoids treating an old choice as a permanent rule.

Q204 How do you mentor engineers without becoming a bottleneck

Explain invariants and decision criteria, pair on unfamiliar work, and provide examples of good changes. Give ownership with clear boundaries and useful feedback. Improve documentation and automated checks so routine decisions do not require your approval.

In an interview, use a real example and distinguish your contribution from the team's work. Do not invent metrics or claim responsibility for every outcome. Explain how another engineer became more independent as a result.

Q205 How do you answer a disagreement or failure question

Describe the context, your reasoning, the other view, the evidence used, and the outcome. For a failure, explain the impact, your actions, and what changed afterward. Own your part without exaggerating blame or presenting yourself as the sole rescuer.

If there were no measured results, say so and describe observable outcomes honestly. A credible explanation of an imperfect decision is stronger than a polished story with unsupported numbers.

Q206 What should you ask the interviewer

Ask about the team's Python runtime and frameworks, deployment model, biggest reliability issues, testing expectations, ownership boundaries, and what success looks like in the first months. Ask which constraints are genuinely difficult rather than asking only which tools are used.

For a senior role, explore decision authority, on-call responsibilities, migration plans, and collaboration with product and platform teams. Their answers help you understand the work and choose relevant examples from your own experience.

20 Recent Python changes and difficult follow ups

Q207 Which Python version changes should you discuss carefully

State the project's supported versions before using new syntax or APIs. Examples in this guide use a Python 3.11 baseline where possible. Python 3.14 has changes around free-threading support, annotation evaluation, and multiprocessing defaults that can affect infrastructure and introspection code. [25]

Do not treat “latest” as a timeless technical specification. Use an explicit version in documentation and CI, and verify dependency compatibility before an upgrade. A feature being available does not mean the application's deployment actually enables it.

Q208 Why does annotation evaluation matter

Python 3.14 changes default annotation behavior toward deferred evaluation and introduces `annotationlib` facilities for introspection. Code that inspects annotations must account for evaluation mode and unresolved names. Evaluating annotations from untrusted code can execute code; annotations are not inherently safe data. [24]

For a typical application, rely on compatible maintained tooling instead of writing a custom annotation evaluator. For a framework author, test introspection across supported versions and document whether forward references are resolved.

Q209 Is match case just a switch statement

Structural pattern matching can match shapes of data and bind names, not merely compare one value against constants. Guards can add conditions. Bare names in patterns can capture values, which surprises developers expecting every name to represent a preexisting constant.

Use it when structural cases make the code clearer, such as parsing a small set of tagged messages. Keep a fallback for unsupported shapes and validate external input. A long match is not automatically better than a dictionary of handlers.

Q210 What are exception groups useful for

An exception group represents multiple related exceptions, which can arise when concurrent tasks fail. `except*` handles matching subgroups rather than treating all failures as one ordinary exception. This is useful with structured concurrency but requires understanding how partial handling works.

Do not flatten all failures into a generic string if callers need to distinguish them. At an API boundary, preserve diagnostic detail internally while returning an appropriate stable error response.

Q211 Are Python operations atomic

Some operations may be indivisible under a particular implementation and execution model, but you should not infer an application guarantee from an experiment or a list of bytecodes. User-defined methods, native calls, runtime versions, and free-threaded builds complicate assumptions.

Protect the whole invariant with documented synchronization or a database operation. For example, “if key is absent, create exactly one external resource” is a multi-step operation even if dictionary insertion itself preserves dictionary integrity.

Q212 Can a Python service guarantee exactly once processing

Only within carefully defined boundaries and assumptions. A database transaction can atomically record a deduplication key and a local business change. A remote side effect across an unreliable network introduces uncertainty unless the remote system supports compatible idempotency or transactional coordination.

Explain what is exactly once: message delivery, handler execution, or externally visible effect. Many practical systems use repeated delivery plus idempotent effects and reconciliation. Avoid promising a stronger guarantee than the components can provide.

21 Worked coding exercises with solutions

These exercises include their contracts, reasoning, code, and boundary checks. Standard-library examples can be run independently unless a fragment is explicitly identified. The web and SQL examples require their stated dependencies or schema. During practice, hide the solution and explain your invariant before writing code.

Q213 Find two indices whose values add to a target

Contract: accept a list of integers and return one pair of distinct indices, or None if no pair exists. When multiple pairs exist, any valid pair is acceptable. Store previously seen values in a dictionary. Before recording the current value, check whether its complement was seen. This order prevents using the same index twice.

```
def two_sum(values, target):
    seen = {}
    for index, value in enumerate(values):
        complement = target - value
        if complement in seen:
            return seen[complement], index
        seen[value] = index
    return None

assert two_sum([2, 7, 11, 15], 9) == (0, 1)
assert two_sum([3, 3], 6) == (0, 1)
assert two_sum([3], 6) is None
assert two_sum([], 0) is None
```

Complexity: average $O(n)$ time and $O(n)$ auxiliary space. A sorting-based two-pointer solution may reduce map storage but must preserve original indices and normally costs $O(n \log n)$. Follow-up: explain how the answer changes if all pairs are required, because the output itself can be quadratic.

Q214 Check a palindrome while ignoring nonalphanumeric code points

Contract: compare alphanumeric code points after lowercase conversion and ignore other code points. This is a common coding-interview contract, not a complete Unicode grapheme-aware specification. Move two pointers inward, skipping ignored characters.

```
def is_palindrome(text):
    left, right = 0, len(text) - 1
    while left < right:
        if not text[left].isalnum():
            left += 1
        elif not text[right].isalnum():
            right -= 1
        elif text[left].lower() != text[right].lower():
            return False
        else:
            left += 1
            right -= 1
    return True
```

```
assert is_palindrome("A man, a plan, a canal: Panama")
assert not is_palindrome("race a car")
assert is_palindrome("")
assert is_palindrome("!!!")
```

Complexity: $O(n)$ time and $O(1)$ auxiliary space under the stated per-code-point contract. The invariant is that every comparable position outside the active interval has already matched. Follow-up: for human-language matching, discuss normalization and case folding rather than silently extending the claim.

Q215 Find the maximum average of a fixed length subarray

Contract: k is an integer and must satisfy $1 \leq k \leq \text{len}(\text{values})$. Values are ordinary finite numbers. Initialize one window sum, then remove the outgoing item and add the incoming item. Compare sums because all windows have the same length.

```
def max_average(values, k):
    if not 1 <= k <= len(values):
        raise ValueError("k must fit the input")
    window = sum(values[i] for i in range(k))
    best = window
    for right in range(k, len(values)):
        window += values[right] - values[right - k]
        best = max(best, window)
    return best / k

assert max_average([1, 12, -5, -6, 50, 3], 4) == 12.75
assert max_average([-5, -2, -8], 1) == -2.0
assert max_average([5], 1) == 5.0
```

Complexity: $O(n)$ time and $O(1)$ auxiliary space. Initializing best to zero would fail when all candidate sums are negative. In Python 3, / already performs true division; multiplying by 1.0 is unnecessary. Follow-up: a streaming input needs a buffer of the most recent k items if old values cannot be indexed.

Q216 Find the longest substring without repeated characters

Contract: return length measured in Python string code points. Track the most recent index of each code point. When a repeat occurs inside the active window, move the left boundary past its previous occurrence. Never move the boundary backward.

```
def longest_unique(text):
    last = {}
    left = best = 0
    for right, char in enumerate(text):
        if char in last:
            left = max(left, last[char] + 1)
        last[char] = right
        best = max(best, right - left + 1)
    return best

assert longest_unique("abcabcbb") == 3
assert longest_unique("bbbbbb") == 1
assert longest_unique("abba") == 2
assert longest_unique("") == 0
```

Complexity: average $O(n)$ time and $O(u)$ space for u distinct code points. The abba test detects a common bug where the left pointer moves backward after seeing an old character outside the current window. Follow-up: return the actual substring by recording the best interval.

Q217 Count subarrays whose sum equals a target

Contract: integers may be negative, and count nonempty contiguous subarrays. If the current prefix is p , every earlier prefix equal to $p - \text{target}$ begins a matching interval. Store frequencies because the same prefix can occur multiple times.

```
def count_target_subarrays(values, target):
    frequencies = {0: 1}
    prefix = total = 0
    for value in values:
        prefix += value
        total += frequencies.get(prefix - target, 0)
        frequencies[prefix] = frequencies.get(prefix, 0) + 1
    return total
```

```
assert count_target_subarrays([1, 1, 1], 2) == 2
assert count_target_subarrays([0, 0], 0) == 3
assert count_target_subarrays([1, -1, 0], 0) == 3
assert count_target_subarrays([], 0) == 0
```

Complexity: average $O(n)$ time and $O(n)$ space. A positive-number sliding window is not generally valid because negative numbers break monotonic sum behavior. Follow-up: returning every matching interval may require output space proportional to the number of matches.

Q218 Compute the integer square root with binary search

Contract: for nonnegative integer n , return the largest integer r such that $r*r \leq n$. Reject negative input. Search the inclusive interval and remember the last feasible midpoint.

```
def integer_sqrt(n):
    if n < 0:
        raise ValueError("n must be nonnegative")
    low, high, answer = 0, n, 0
    while low <= high:
        middle = (low + high) // 2
        if middle * middle <= n:
            answer = middle
            low = middle + 1
        else:
            high = middle - 1
    return answer
```

```
assert integer_sqrt(0) == 0
assert integer_sqrt(1) == 1
assert integer_sqrt(8) == 2
assert integer_sqrt(10**12) == 10**6
```

Complexity: $O(\log n)$ iterations, with integer arithmetic costs depending on operand size. Auxiliary state uses a constant number of integers. In real application code, `math.isqrt` is the standard-library choice. Follow-up: a floating approximation requires a tolerance and convergence argument rather than reusing the same stopping rule.

Q219 Merge overlapping intervals

Contract: each interval is a pair (start, end) with $\text{start} \leq \text{end}$. Treat touching closed intervals as mergeable. Sort by start, then either extend the last merged interval or begin a new one. The input list is not mutated.

```
def merge_intervals(intervals):
    if any(start > end for start, end in intervals):
        raise ValueError("invalid interval")
    merged = []
    for start, end in sorted(intervals):
        if not merged or start > merged[-1][1]:
            merged.append([start, end])
        else:
            merged[-1][1] = max(merged[-1][1], end)
    return merged

assert merge_intervals([(1, 3), (2, 6), (8, 10)]) == [
    [1, 6], [8, 10]
]
assert merge_intervals([(1, 2), (2, 3)]) == [[1, 3]]
assert merge_intervals([]) == []
```

Complexity: $O(n \log n)$ time and $O(n)$ space including sorting and output. The contract accepts a reusable sequence; if it accepted a one-pass iterator, the validation pass would consume it. Follow-up: half-open scheduling intervals may need different rules for endpoints that touch.

Q220 Validate balanced brackets

Contract: accept only the six bracket characters and return false for other characters. Push opening brackets. On a closing bracket, the stack top must contain the corresponding opening bracket.

```
def valid_brackets(text):
    pairs = {"(": ")", "[": "]", "{": "}"}
    openings = set(pairs.values())
    stack = []
    for char in text:
        if char in openings:
            stack.append(char)
        elif char in pairs:
            if not stack or stack.pop() != pairs[char]:
                return False
        else:
            return False
    return not stack

assert valid_brackets("([]{})")
assert not valid_brackets("([)]")
assert not valid_brackets("(")
assert valid_brackets("")
```

Complexity: $O(n)$ time and $O(n)$ space in the worst case. Counting opening and closing brackets is insufficient because nesting order matters. Follow-up: parsing real source code requires handling quoted strings, escapes, and comments rather than applying this function directly.

Q221 Find an unweighted shortest path with BFS

Contract: the graph is an adjacency mapping of hashable nodes to neighbor iterables. Return one shortest path or None. Record predecessors when enqueueing nodes, then reconstruct the path after reaching the goal.

```
from collections import deque
```

```
def shortest_path(graph, start, goal):
    pending = deque([start])
    parent = {start: None}
    while pending:
        node = pending.popleft()
        if node == goal:
            path = [node]
            while path[-1] != start:
                path.append(parent[path[-1]])
            return path[::-1]
        for neighbor in graph.get(node, ()):
            if neighbor not in parent:
                parent[neighbor] = node
                pending.append(neighbor)
    return None
```

```
graph = {"a": ["b", "c"], "b": ["d"], "c": ["d"]}
assert shortest_path(graph, "a", "d") == ["a", "b", "d"]
assert shortest_path(graph, "a", "a") == ["a"]
assert shortest_path(graph, "d", "a") is None
```

Complexity: $O(V + E)$ over the reachable graph, with $O(V)$ auxiliary space. Copying an entire path for every queued neighbor is simpler but can add unnecessary memory. Follow-up: weighted edges require a different algorithm; BFS is correct when every edge has the same cost.

Q222 Return the k largest values from a stream

Contract: k is nonnegative; retain duplicates and return results in descending order. Maintain a min-heap of at most k items. Its smallest value is the threshold for entering the current top group.

```
import heapq
```

```
def largest_k(values, k):
    if k < 0:
        raise ValueError("k must be nonnegative")
    if k == 0:
        return []
    heap = []
    for value in values:
        if len(heap) < k:
            heapq.heappush(heap, value)
        elif value > heap[0]:
            heapq.heapreplace(heap, value)
    return sorted(heap, reverse=True)
```

```
assert largest_k(iter([5, 1, 9, 9, 2]), 3) == [9, 9, 5]
assert largest_k([2, 1], 5) == [2, 1]
assert largest_k([], 2) == []
```

Complexity: $O(n \log k + k \log k)$ when $1 < k \leq n$, and $O(k)$ storage, with the actual heap bounded by the number of items. Follow-up: for records, include a deterministic tie-breaker so equal priorities do not force comparison of incompatible payloads.

Q223 Implement a small least recently used cache

Contract: fixed positive capacity; get returns a supplied default when absent; reads and writes mark an entry recently used. OrderedDict makes movement and oldest-entry removal explicit. This example is local and not thread-safe.

```
from collections import OrderedDict
```

```
class LRUCache:
    def __init__(self, capacity):
        if capacity <= 0:
            raise ValueError("capacity must be positive")
        self.capacity = capacity
        self.data = OrderedDict()

    def get(self, key, default=None):
        if key not in self.data:
            return default
        self.data.move_to_end(key)
        return self.data[key]

    def put(self, key, value):
        self.data[key] = value
        self.data.move_to_end(key)
        if len(self.data) > self.capacity:
            self.data.popitem(last=False)
```

```
cache = LRUCache(2)
cache.put("a", 1)
cache.put("b", 2)
assert cache.get("a") == 1
cache.put("c", 3)
assert cache.get("b") is None
```

Operations are average $O(1)$, with $O(\text{capacity})$ storage. A production cache needs byte limits, expiry, concurrency, ownership of mutable values, and observability. Follow-up: describe a dictionary plus doubly linked list implementation, and explain why a count limit alone cannot bound memory when values vary greatly in size.

Q224 Find the minimum number of coins for an amount

Contract: coin denominations are positive integers, supply is unlimited, amount is nonnegative, and return -1 if impossible. Let `best[x]` be the smallest number of coins making amount `x`. Each transition considers the final coin.

```
def minimum_coins(coins, amount):
    if amount < 0 or any(coin <= 0 for coin in coins):
        raise ValueError("invalid amount or denomination")
    denominations = set(coins)
    best = [0] + [amount + 1] * amount
    for subtotal in range(1, amount + 1):
        for coin in denominations:
            if coin <= subtotal:
                best[subtotal] = min(
                    best[subtotal], best[subtotal - coin] + 1
                )
    return -1 if best[amount] > amount else best[amount]
```

```
assert minimum_coins([1, 2, 5], 11) == 3
assert minimum_coins([2], 3) == -1
assert minimum_coins([], 0) == 0
assert minimum_coins([1, 3, 4], 6) == 2
```

Complexity: $O(\text{amount} \times \text{distinct denominations})$ time and $O(\text{amount})$ space. The last test shows why greedily taking the largest coin can fail. Follow-up: a huge amount may make this pseudo-polynomial algorithm impractical even when the input has few digits.

Q225 Yield fixed size batches from an iterable

Contract: consume a one-pass iterable, yield tuples of up to size items, and reject nonpositive sizes. The last batch can be shorter. A generator keeps only the current batch in memory.

```
from itertools import islice
```

```
def batches(iterable, size):
    if size <= 0:
        raise ValueError("size must be positive")
    iterator = iter(iterable)
    while True:
        batch = tuple(islice(iterator, size))
        if not batch:
            return
        yield batch
```

```
assert list(batches(range(5), 2)) == [(0, 1), (2, 3), (4,)]
assert list(batches([], 2)) == []
```

Complexity: $O(n)$ total time and $O(\text{size})$ working memory, excluding retained output. Validation inside a generator body happens when iteration starts. Follow-up: if a caller requires immediate validation on the function call, use a non-generator outer function that validates and returns an inner generator.

Q226 Write a decorator that measures synchronous execution

Contract: preserve metadata, return values, and exceptions, and report elapsed time through an injected callback. Use a monotonic performance clock. The callback must be safe and must not raise, or it could mask the wrapped result or exception.

```
from functools import wraps
from time import perf_counter

def timed(report):
    def decorate(function):
        @wraps(function)
        def wrapper(*args, **kwargs):
            started = perf_counter()
            try:
                return function(*args, **kwargs)
            finally:
                report(function.__name__, perf_counter() - started)
        return wrapper
    return decorate

observations = []

@timed(lambda name, seconds: observations.append((name, seconds)))
def add(left, right):
    return left + right

assert add(2, 3) == 5
assert observations[0][0] == "add"
assert observations[0][1] >= 0
```

The wrapper adds constant bookkeeping overhead. Follow-up: an async version must itself be async and await the wrapped call inside the protected interval. Also discuss metric-label cardinality before attaching arbitrary argument values to observations.

Q227 Process async work with a bounded queue

Contract: input is a finite synchronous iterable; operation is an async callable; positive worker and buffer counts are required. This example uses Python 3.11 TaskGroup and fails the group if a worker fails. It does not retain results, so work-state memory is bounded by the queue and worker count. The input source itself may have its own memory cost.

```
import asyncio

async def consume_bounded(items, operation, workers=3, buffer=6):
    if workers <= 0 or buffer <= 0:
        raise ValueError("workers and buffer must be positive")
    queue = asyncio.Queue(maxsize=buffer)
    stop = object()

    async def produce():
        for item in items:
            await queue.put(item)
        for _ in range(workers):
            await queue.put(stop)

    async def consume():
        while True:
            item = await queue.get()
            try:
                if item is stop:
                    return
                await operation(item)
            finally:
                queue.task_done()

    async with asyncio.TaskGroup() as group:
        group.create_task(produce())
        for _ in range(workers):
            group.create_task(consume())

async def example():
    completed = []
    async def record(item):
        await asyncio.sleep(0)
        completed.append(item * 2)
    await consume_bounded(range(5), record)
    assert sorted(completed) == [0, 2, 4, 6, 8]

asyncio.run(example())
```

Do not use this producer unchanged for a blocking file or network iterator in an event loop. The sample also does not guarantee output order or durable delivery. Follow-up: add per-item deadlines and decide whether one failure should cancel all work or be recorded for later retry.

Q228 Implement bounded retries without hiding permanent failures

Contract: retry only exceptions explicitly supplied by the caller, use a positive attempt limit, and let the final exception propagate. Backoff includes jitter. Sleep and randomness are injectable so tests can be deterministic. The caller must ensure the operation is safe to repeat.

```
import random
import time

def retry_call(operation, retryable, attempts=3, base=0.1,
               cap=2.0, sleep=time.sleep, random_value=random.random):
    if attempts <= 0 or base < 0 or cap < 0:
        raise ValueError("invalid retry policy")
    for attempt in range(attempts):
        try:
            return operation()
        except retryable:
            if attempt + 1 == attempts:
                raise
            delay = min(cap, base * (2 ** attempt))
            sleep(delay * random_value())

calls = []
delays = []

def flaky():
    calls.append(1)
    if len(calls) < 3:
        raise TimeoutError("temporary")
    return "ok"

assert retry_call(
    flaky, (TimeoutError,), sleep=delays.append,
    random_value=lambda: 0.5
) == "ok"
assert len(calls) == 3
assert len(delays) == 2
```

This helper bounds attempts, not total elapsed time. Each call still needs a timeout and a total deadline in a production client. Follow-up: account for server retry instructions and explain why retrying a POST after a timeout can duplicate a remote side effect.

Q229 Build a validated calculation endpoint

Contract: demonstrate request validation and a response schema with FastAPI and Pydantic v2. This example calculates a subtotal and has no database or external side effect. Amounts use integer cents to avoid binary floating-point rounding. Install compatible FastAPI and Pydantic versions in a project environment before running it.

```
from fastapi import FastAPI
from pydantic import BaseModel, ConfigDict, Field

app = FastAPI()

class QuoteRequest(BaseModel):
    model_config = ConfigDict(extra="forbid", strict=True)
    unit_price_cents: int = Field(ge=0, le=10_000_000)
    quantity: int = Field(ge=1, le=1000)

class QuoteResponse(BaseModel):
    subtotal_cents: int

@app.post("/quotes", response_model=QuoteResponse)
def quote(body: QuoteRequest) -> QuoteResponse:
    return QuoteResponse(
        subtotal_cents=body.unit_price_cents * body.quantity
    )
```

Example request: {"unit_price_cents": 1250, "quantity": 3}. Expected successful body: {"subtotal_cents": 3750}. Negative prices, zero quantities, extra fields, and string values in strict integer fields should fail validation. A real checkout must use trusted catalog prices instead of accepting the user's claimed price, and it needs authentication, authorization, and durable order handling.

Q230 Deduplicate source records with SQL

Contract: select the newest version per source record using a source version and an ingestion sequence as a deterministic tie-breaker. This SQL fragment assumes a staging_records table with the named columns. Adapt types and transaction handling to the target database.

```
WITH ranked AS (
    SELECT
        record_id,
        source_version,
        payload,
        ROW_NUMBER() OVER (
            PARTITION BY record_id
            ORDER BY source_version DESC, ingestion_sequence DESC
        ) AS position
    FROM staging_records
)
SELECT record_id, source_version, payload
FROM ranked
WHERE position = 1;
```

Expected behavior: if record 7 has versions 2 and 4, return version 4. If version 4 appears twice, use the larger ingestion sequence. A tie-breaker is only meaningful if the source contract permits it; conflicting payloads for the same authoritative version may instead require rejection. Follow-up: merge the selected result into the target only when the incoming version is newer, and handle deletion records explicitly.

22 Worked Python system design interviews

System design is not a contest to name the most infrastructure. Begin with the user action, required guarantees, and load assumptions. Then choose data ownership, APIs, failure handling, and only the components needed to meet the requirements. The following ten cases are original interview designs with illustrative assumptions, not production benchmark results.

Q231 Design a URL shortening service

Requirements: users create short links and visitors resolve them. Clarify custom aliases, expiration, link editing, analytics, abuse handling, and whether links are public. Assume, for this exercise, 100 new links per second and 10,000 redirects per second, with read traffic dominating. Define an availability and latency target before deciding whether multiple regions are needed.

API and data: POST /links accepts a target URL and optional expiry; GET /{code} resolves it. Store code, destination, owner, creation time, expiry, and status. Enforce a unique code in the database. Generate a sufficiently large random code or encode a generated identifier; retry collisions under the unique constraint. Never rely solely on a pre-insert existence check.

Request path: a Python API validates the allowed URL scheme, checks ownership rules, and stores the mapping. Redirect requests read a cache and fall back to the database. Choose a redirect status and cache policy consistent with whether links can change. A permanent redirect can be cached by clients and make later edits ineffective for those clients.

Scale and estimate: at 100 writes per second, daily creations are 8.64 million if the rate is sustained. At an assumed 300 bytes of logical data per row, that is about 2.6 GB per day before indexes, replication, and storage overhead. State that sustained traffic may differ from a peak rate. The estimate tells you to discuss retention and growth rather than assuming the table remains small forever.

Failures and trade-offs: cache failure should fall back under controlled load, not overload the database immediately. Popular codes need efficient caching and stampede protection. Analytics should be asynchronous so redirect latency does not depend on an analytics write. If analytics events may be lost, say whether that is acceptable. Reject dangerous schemes, rate-limit creation, and provide an abuse-removal path.

Python decisions: a small FastAPI, Flask, or Django endpoint can serve the mapping; database and cache behavior are likely more important than framework choice. Reuse clients with appropriate lifetimes. Avoid storing the authoritative map in a global dictionary across workers.

Senior follow-up: explain how disabled links invalidate caches and how regional replicas affect the time until a new link is visible. If immediate global visibility is required, justify the replication and read-routing cost. Measure redirect p95/p99 latency, cache hit rate, missing-code rate, database fallback load, and abuse response time.

Q232 Design a distributed rate limiter

Requirements: enforce per-tenant API limits across multiple Python service replicas, with a small allowed burst. Ask whether rejected requests must be exact, whether traffic must remain available when the limiter fails, and whether the policy applies to every route or weighted operations.

Algorithm: a token bucket refills capacity over time and consumes tokens per request. A fixed window is simpler but permits boundary bursts. A sliding log is precise but can consume more memory. Choose one based on required accuracy and cost rather than calling every algorithm interchangeable.

State and atomicity: store bucket tokens and last-refill time in a shared low-latency store. Perform refill, comparison, decrement, and expiry as one atomic operation supported by that store. A Python read followed by a Python write races across replicas. Use a consistent time source for shared calculations and cap refill at bucket capacity.

API behavior: rejected requests return a suitable throttling response and a meaningful retry indication. Scope keys by tenant and policy version; consider per-user sublimits when one user can exhaust a tenant allocation. Avoid high-cardinality metrics for every raw key while still retaining useful sampled diagnostics.

Failure decision: fail-open preserves availability but may exceed contractual limits; fail-closed protects resources but can turn limiter failure into a full outage. A bounded local fallback can reduce impact while sacrificing exact global accounting. State which business requirement justifies the choice.

Python implementation: middleware or a dependency calls the limiter with a strict timeout and reuses a client pool. Keep the algorithm's atomic portion in the shared store rather than protected only by threading.Lock, which coordinates one process at most.

Senior follow-up: a single huge tenant can become a hot key. Local token allocations reduce central traffic but can over-admit during failure or reallocation. Quantify the maximum overshoot you are willing to accept. Test simultaneous requests from separate processes, expiry boundaries, clock changes, and shared-store failure.

Q233 Design a durable notification system

Requirements: deliver email, SMS, or push notifications from application events. Clarify urgency, provider limits, deduplication, user preferences, unsubscribe behavior, templates, and audit needs. Acceptance of a notification request is not the same as successful delivery to a device.

Data and flow: write a notification intent and outbox event in a database transaction. A publisher places work on channel-specific queues. Python workers check current eligibility, render a versioned template, and call a provider. Store operation state, attempts, provider ID, and relevant timestamps. Use provider callbacks or polling to update delivery status when supported.

Idempotency: use an event ID plus recipient and channel as a logical deduplication key where appropriate. A worker can crash after the provider accepts the message but before local status is saved. Use the provider's idempotency facility if available; otherwise document possible duplicates and reconcile using provider identifiers. An internal unique row alone does not eliminate this remote uncertainty.

Retries and ordering: retry transient provider failures with jitter and bounded deadlines. Route permanent failures to an inspectable terminal state. Separate urgent transactional notifications from bulk campaigns so a campaign cannot block a password-reset message. Decide whether ordering is required per recipient; enforcing global order is usually unnecessary.

Capacity: if a provider allows 200 requests per second and arrivals sustain 300, the backlog grows by 100 per second even with unlimited workers. Worker autoscaling cannot overcome that provider limit. Add channel routing, admission policies, or negotiated capacity rather than only adding replicas.

Security and operations: templates must avoid unsafe content injection; callback signatures must be verified; logs should minimize message content. Track time to provider acceptance separately from actual delivery. Alert on oldest pending age, rejection rate, provider latency, and duplicate reports.

Senior follow-up: deleting an account or changing notification preferences while work is queued creates a policy timing question. Decide whether eligibility is evaluated at creation, sending, or both, and preserve enough audit data to explain the choice.

Q234 Design a large file processing service

Requirements: users upload files, processing may take minutes, and results must survive deployments. Clarify maximum compressed and expanded size, accepted formats, tenant isolation, cancellation, retention, and whether partial results are useful.

Upload flow: the API creates an upload record and an authorized upload destination in object storage. The client uploads directly when appropriate. A finalize step verifies ownership, object metadata, and completion before marking the file ready. An object existing at a path is not proof that it is a valid or authorized submission.

Processing flow: a database transaction records a job and outbox event. Workers fetch the object, validate its type and resource limits, process bounded chunks, write versioned output, and update durable progress. Use states such as uploaded, queued, running, succeeded, failed, and cancelled, with clearly permitted transitions.

Recovery: assign a lease or heartbeat to running work. A replacement worker can reclaim expired work, but old workers may still be executing. Use versioned state transitions or fencing where stale workers could overwrite results. Make output creation deterministic or uniquely versioned so retries do not corrupt a shared result.

Resource controls: compressed archives and malformed documents can consume much more CPU or memory than their upload size suggests. Restrict expanded size, nesting, processing duration, and concurrent jobs per tenant. Isolate processing from latency-sensitive API workers.

Python decisions: use streaming parsers where available and process CPU-heavy transformations in suitable worker processes. Keep application code from loading an entire multi-gigabyte file into memory. A local temporary file may be useful during processing but must not be the only durable copy.

Senior follow-up: progress percentages can be misleading when processing stages have different costs. Report stage and completed units where possible. Test crash recovery after output write but before status commit, duplicate finalize requests, cancellation during a chunk, and missing source objects.

Q235 Design inventory reservation for checkout

Requirements: reserve a limited quantity without overselling, support duplicate client retries, and release expired reservations. Clarify whether backorders are allowed, reservation duration, payment ordering, and whether inventory spans warehouses. Money and inventory effects require stronger correctness than approximate analytics.

Data model: products or stock rows, reservations, reservation items, orders, and idempotency records. Store reservation state and expiry. Enforce positive quantities and unique logical operation keys. The API must look up trusted prices and product rules rather than accept client-supplied totals as authoritative.

Transaction: atomically reserve only if available quantity is sufficient, using a conditional update or suitable row locks. Create the reservation and operation record in the same transaction. For multiple items, lock in a stable order to reduce deadlocks. If one required item cannot be reserved, roll back the transaction according to the all-or-nothing contract.

Payment boundary: do not hold a database transaction open while waiting indefinitely on a payment provider. Use an explicit workflow: reserve, request payment with a stable remote idempotency key, confirm or release. A payment timeout is an unknown outcome until the provider is checked. Do not immediately create a new payment operation.

Expiry: a background process releases expired reservations using conditional state transitions so the release happens once. Confirmation and expiry can race; only one permitted transition should win. A cache expiry alone must not serve as the authoritative inventory release mechanism.

Scale: a highly popular product can create row contention. Partitioning by warehouse or using carefully allocated inventory buckets can improve throughput but adds reconciliation and allocation complexity. Begin with database correctness and measure contention before distributing the stock counter.

Senior follow-up: explain recovery when payment succeeds after the reservation expires. Options depend on policy: attempt a new reservation, refund, or route for resolution. State that these are visible business decisions, not merely exception-handling details. Monitor oversell violations, lock waits, expired reservations, duplicate requests, and unresolved payment states.

Q236 Design a resumable data ingestion pipeline

Requirements: copy changing records from a paginated source API into analytics storage. Clarify source ordering, update timestamps, deletion representation, consistent snapshot support, rate limits, and target freshness. The pipeline must be rerunnable and reconcile its results.

Extraction: create a run manifest containing source identity, schema version, watermark interval, start time, and object list. Stream bounded JSONL batches into durable object storage, optionally compressed. Persist checksums and record counts. If the source supports an upper-bound filter, capture one so a run has a defined interval.

Transformation and loading: validate field types and map source IDs to a stable target schema. Load into an isolated staging area, deduplicate by source key and version, then merge into the target with a newer-version rule. Preserve invalid records in a controlled rejection location or fail the batch according to the agreed data-quality policy.

Checkpoint rule: advance the committed watermark only after the target merge and reconciliation succeed. Reusing the same input objects should produce the same target state. If the source only offers imperfect timestamp pagination, overlap extraction windows and deduplicate rather than pretending the watermark guarantees no gaps.

Reconciliation: compare extraction counts, written records, parsed records, accepted records, rejected records, and merged outcomes. Counts alone cannot detect all corruption; add key coverage, checksums where meaningful, and sampled field comparisons. Track deletions and out-of-order updates separately.

Python decisions: use a bounded request client with retries for appropriate transient failures, stream compression correctly, and avoid materializing all source pages. Separate extraction, transformation, and target writing so each can be tested against a recorded batch.

Senior follow-up: schema drift can introduce new fields or change existing types. Decide which changes are compatible, which require quarantine, and who approves schema evolution. Explain how a historical backfill coexists with incremental ingestion without letting older data overwrite newer target state.

Q237 Design a multi tenant document question answering service

Requirements: users upload documents and ask questions; answers must respect document permissions and reference supporting passages. Clarify document formats, update frequency, maximum size, supported languages, retention, answer latency, and cost budget. Define what happens when the available documents do not contain an answer.

Ingestion: authenticate the upload, store the source object, and queue extraction. Chunk content with document, tenant, permission, version, and location metadata. Generate embeddings using a recorded

model version, write the search index, then atomically mark the document version queryable when its required stages succeed.

Query path: authenticate the caller, derive allowed scope, retrieve only authorized candidates, rerank if justified, build a bounded context, and call the model. Validate the response structure and attach references to the actual retrieved passages. Keep retrieved text as evidence, not as authority to change the system's instructions or tool permissions.

Data boundaries: document text, embeddings, metadata, conversation history, and caches all require tenant isolation. Filtering only after unrestricted top-k retrieval can damage recall and can expose unauthorized content if another component logs or forwards it. Apply access constraints before information crosses the protected boundary.

Reliability: document processing needs idempotent jobs and deletion propagation. A deleted document must become inaccessible promptly even if physical vector deletion is delayed. A query cache must incorporate permission scope, document version, and relevant model or prompt versions. Streaming should identify partial versus completed answers.

Evaluation: measure retrieval recall on representative questions, citation support, answer correctness, refusal when evidence is absent, latency, and cost. Keep an evaluation set with difficult negatives and cross-tenant access tests. A lower latency is not a success if retrieval quality deteriorates.

Senior follow-up: upgrading embeddings usually requires re-embedding and a versioned index strategy. Plan dual indexing or a controlled cutover, compare retrieval quality, and avoid mixing incompatible vector spaces. For tool-using extensions, deterministic permission checks must authorize every side effect independently of model output.

Q238 Design a real time chat service

Requirements: users send messages to conversations, reconnect after disconnects, and see an agreed delivery order. Clarify group sizes, attachments, history retention, offline delivery, read receipts, editing, and moderation. Distinguish sent, persisted, delivered, and read states.

Architecture: Python ASGI connection handlers authenticate clients and authorize conversation access. Persist messages with a conversation ID, sender ID, client-generated deduplication ID, server sequence or ordering key, and timestamp. Publish durable message events and fan them out to connected recipients through a shared transport.

Ordering and duplicates: assign ordering within a conversation rather than promising a global order across the whole product. A reconnecting client sends its last acknowledged position and receives missing history. Deduplicate a retried send by sender and client message ID. Make the same message appear once in the user interface even if transport delivery repeats.

Presence: presence is approximate and ephemeral because clients and networks fail without a clean disconnect. Store heartbeats with expiry and present semantics such as recently active if strict online status cannot be guaranteed. Do not use presence as authorization or durable message state.

Backpressure: slow clients need bounded outgoing buffers. If a buffer fills, choose to disconnect and let the client replay history rather than consume unlimited server memory. Attachments belong in object storage with controlled access, not directly in every message queue payload.

Python decisions: avoid synchronous database and network calls in the connection event loop. A WebSocket connection belongs to one process, but room membership and message delivery must work across processes. Sticky routing may help connection continuity but does not replace shared durable history.

Senior follow-up: decide how permission changes affect already connected users and how message deletion propagates to caches and clients. Test reconnect storms, duplicate publishes, slow recipients, and a fan-out worker crashing after persistence but before delivery.

Q239 Design a job scheduler for recurring Python tasks

Requirements: execute recurring schedules, track outcomes, support retries, and survive scheduler failure. Clarify timezone semantics, missed runs, overlapping executions, maximum lateness, and whether users can change a schedule while a run is active.

Data model: schedules contain expression or interval, timezone, enabled state, and next due time. Runs contain schedule ID, scheduled occurrence, state, attempts, lease, and output metadata. A unique constraint on the schedule and scheduled occurrence prevents two schedulers from creating the same logical run.

Coordination: multiple scheduler instances can claim due work using an appropriate locking or leasing mechanism. They create durable run records and outbox events, then workers execute jobs. Leases help recover from failure, but stale workers need guarded state updates and idempotent effects.

Calendar rules: a daily local-time job behaves differently from an every-24-hours interval. Daylight saving transitions may skip or repeat local times. Define the policy explicitly and store the scheduled occurrence so a delayed worker does not infer it from its current clock.

Overlap and retries: decide whether a new occurrence may run while an older one is active. Retry attempts belong to the same logical occurrence, while a later scheduled run is different work. For catch-up after an outage, choose whether to run all missed occurrences, only the latest, or none.

Python decisions: keep scheduling separate from task business code. Task functions should accept an explicit data interval and operation ID and should not rely on module globals for coordination. A local `while True: sleep(...)` loop is insufficient when durability and multiple replicas matter.

Senior follow-up: ask how the system handles a job that never returns, a clock correction, and a schedule edited during dispatch. Monitor due-to-start delay, oldest pending run, execution time, duplicate suppression, and expired leases.

Q240 Design a high traffic Python API and capacity plan

Requirements: define the request mix, payload size, p95 and p99 latency targets, write consistency, and dependency limits. Assume an illustrative average of 2,000 requests per second and average in-system time of 0.2 seconds under steady conditions. Little's Law suggests about 400 average in-flight requests across the measured system boundary; this is not a worker-count formula.

Baseline: use stateless API replicas, a production application server, bounded database and HTTP pools, and observability. Choose async when waiting concurrency and compatible libraries justify it. Separate CPU-intensive jobs from latency-sensitive handlers. Measure a single replica under representative load before projecting fleet size.

Capacity reasoning: if one replica sustains 250 requests per second while meeting the target under the test conditions, eight replicas cover the illustrative average with no spare capacity. Add headroom for peaks, one or more failures, deploy overlap, and nonlinear downstream contention. Do not assume doubling replicas doubles throughput when a shared database is already saturated.

Latency budget: divide the end-to-end deadline across queueing, validation, business code, database work, downstream calls, and response serialization. Parallel independent calls can reduce critical-path time but increase dependency load. Tail behavior is important when an endpoint fans out to many dependencies.

Overload: reject or defer work before memory and connection pools collapse. Set bounded queue lengths and timeouts. Use per-tenant fairness and separate pools for workloads with different costs. Autoscaling reacts after load changes and cannot substitute for immediate admission control.

Validation: run a ramp test, a sustained test, a burst test, and failure tests for one replica and one dependency. Check correctness, resource growth, cancellation, and recovery, not just achieved requests per second. Load generators must be able to supply the intended traffic without becoming the bottleneck.

Senior follow-up: explain when caching, read replicas, partitioning, or service extraction is justified by measurements. Present the remaining bottleneck and uncertainty after each improvement. A strong answer ends with a plan to validate assumptions, not an unsupported guarantee.

23 Production debugging interview scenarios

Q241 A fast endpoint becomes slow as the database grows

Investigate whether query plans changed, an index is missing, offsets became deep, or a relation access pattern creates repeated queries. Compare rows scanned, rows returned, execution time, and pool waiting. Verify realistic filters and parameter values rather than running only a tiny local query.

Fix the demonstrated cause. An index may help one query but add write cost; cursor pagination may address deep-offset work; eager loading may remove N plus one calls. Test with representative data and verify that response semantics stay the same. Add a targeted query or latency regression check where it provides reliable feedback.

Q242 Memory keeps growing after every batch

Determine whether growth is retained Python objects, native allocations, buffering, or allocator behavior. Inspect references held by caches, result lists, closures, logging queues, and exception objects. Ensure batches and sessions are released and that downstream code does not collect a generator's entire output.

Compare allocation snapshots across controlled batches and observe whether memory stabilizes after a warm-up period. A process RSS graph alone does not prove a leak. Bound intentional caches and consider separate worker lifetimes for third-party native leaks while pursuing the underlying fix.

Q243 An async API handles only one request at a time

Look for blocking HTTP clients, synchronous database access, `time.sleep`, CPU-heavy loops, or serialized access to a shared resource in the event loop. Trace the request and inspect event-loop lag. Confirm the server and worker configuration and the dependencies' execution model.

Use a compatible async client or explicitly offload suitable blocking I/O. Move substantial CPU work to a suitable compute path. Also check test-client and load-generator behavior; sending requests sequentially cannot demonstrate server concurrency. Increasing workers can contain symptoms but may leave the underlying blocking path unresolved.

Q244 Customers receive duplicate notifications

Find the logical operation IDs, broker deliveries, worker attempts, and provider responses. Determine whether duplication originates at request creation, message redelivery, a retry after timeout, or multiple schedulers. Do not infer that the broker is broken merely because one message was handled twice.

Add or correct a durable deduplication key and provider idempotency where available. Reconcile uncertain outcomes before resending. Test a worker crash after provider acceptance but before acknowledgement because that is a key failure window. Explain any remaining duplicate risk when the provider cannot deduplicate.

Q245 Stock becomes negative during a flash sale

Look for a read-modify-save sequence outside adequate transaction control, missing positive-quantity validation, or expiry logic releasing reservations twice. Reproduce with separate concurrent transactions and forced overlap.

Use an atomic conditional update or appropriate locking, enforce database constraints where possible, and make reservation transitions idempotent. A process-local lock is insufficient across replicas. Inspect historical violations and reconcile affected orders rather than assuming the code fix corrects already inconsistent records.

Q246 A deployment causes database connection exhaustion

Count pools across all old and new replicas, worker processes, background jobs, and administration tools. Rolling deployment overlap can temporarily multiply connection demand. Check leaked sessions and long transactions as well as configured pool maxima.

Reduce concurrency or pool limits to restore service, then size pools against the database's capacity and expected waiting. Reuse engines appropriately and scope sessions correctly. Measure connection wait time and transaction duration. Adding more replicas during the incident may worsen the problem.

Q247 One user sees another user data

Treat this as an authorization and data-isolation incident. Identify affected endpoints, tenants, caches, and time range while limiting further exposure. Check queryset scoping, object permission checks, cache keys, signed-link scope, and reused mutable request state.

A fix must cover all representations and access paths, including list endpoints, exports, background jobs, and search indexes. Add cross-user and cross-tenant tests. Follow the organization's incident handling and notification process rather than quietly clearing the cache and assuming no further action is needed.

Q248 A data pipeline has the right row count but wrong values

Check field mapping, duplicate keys, out-of-order updates, timezone conversion, null handling, merge cardinality, and schema drift. Counts can remain unchanged when the wrong version overwrites the correct row. Compare source keys and versions and sample transformed fields.

Restore or replay from preserved raw inputs after correcting the transformation or merge rule. Keep the rerun idempotent and protect newer target versions. Improve reconciliation with version coverage, meaningful checksums, and domain constraints in addition to counts.

Q249 Tests pass locally but fail in CI

Compare interpreter and dependency versions, environment variables, filesystem case behavior, locale, timezone, process start method, and available services. Check tests that rely on execution order, shared state, wall-clock sleeps, or network access.

Reproduce from the same built environment or a clean install. Make time, randomness, and external dependencies controllable when relevant. Do not merely rerun until green or increase every timeout. A flaky test can be evidence of a production race rather than only a testing inconvenience.

Q250 The service becomes slower after adding more workers

Measure CPU contention, memory pressure, database connections, cache contention, and context switching. More workers may multiply model memory or overwhelm a shared dependency. Container CPU quotas can make a host-based worker-count formula inappropriate.

Reduce to a measured baseline, vary one concurrency setting at a time, and track throughput and tail latency under the same workload. Identify the saturated resource and address it directly. Concurrency is a capacity control, not a guarantee of parallel speedup.

24 Practice plans mock interviews and assessment

A twelve week preparation plan

Week 1: learn Chapters 1 and 2 and write small programs using input, functions, loops, and collections. Explain the difference between a returned value and printed output. Practice empty inputs and negative values.

Week 2: study identity, mutation, arguments, scope, and copying. Predict outputs before running examples. Write tests demonstrating the mutable-default and nested-copy traps, then explain the fixes without relying on memorized phrases.

Week 3: study iterators, generators, exceptions, files, and context managers. Build a small streamed file transformer. Define encoding, malformed-record handling, and resource cleanup.

Week 4: study classes, dataclasses, typing, imports, and packaging. Refactor the transformer into a small importable project with an entry point, explicit dependencies, and focused tests.

Week 5: solve the first ten coding exercises from a blank editor. Explain the invariant and complexity before optimizing. Compare at least two optimized solutions against simple reference implementations on small inputs.

Week 6: study HTTP, authentication, authorization, SQL, and transactions. Design a small task or inventory API. Write cross-user access tests and identify which rules need database constraints.

Week 7: choose one primary framework and implement the API. Study the other framework chapters for comparisons. Add validation, response schemas, pagination, database lifetime management, and integration tests.

Week 8: study concurrency and profiling. Demonstrate a blocking async mistake, then correct it. Build a bounded worker example and measure behavior under slow dependencies and cancellation.

Week 9: add a durable job workflow with idempotency and explicit status. Practice explaining crash windows and how an outbox changes them. Introduce duplicate delivery deliberately and verify the intended effect happens once within the defined boundary.

Week 10: study deployment, observability, security, and incidents. Build a repeatable runtime configuration, define readiness and liveness behavior, and write a short rollback plan. Investigate one slow-query and one memory-retention scenario.

Week 11: complete five system design cases aloud. Spend the first minutes clarifying requirements and the final minutes on failure handling and validation. Write down assumptions and challenge them after each session.

Week 12: run the mock interviews below. Review weak areas, repeat the relevant coding problems, and prepare truthful project stories about trade-offs, failures, and collaboration. Prioritize clear reasoning over learning another library name at the last moment.

A beginner mock interview lasting forty five minutes

First 10 minutes: Q002, Q007, Q017, and Q018. Explain variables, truthiness, lists, tuples, and mutation using small examples. Next 20 minutes: solve Q214 or Q215 and test the smallest input. Next 10 minutes: discuss Q031 and Q043. Final 5 minutes: explain what you would improve after getting a correct solution.

Assessment: the candidate should understand the input contract, write a correct loop, return a result, and explain a basic boundary case. Do not demand distributed systems expertise from a beginner role.

A junior backend mock interview lasting sixty minutes

First 10 minutes: Q030, Q039, Q063, and Q110. Next 20 minutes: solve Q216 or Q220. Next 20 minutes: design a small authenticated CRUD endpoint in the candidate's preferred framework, including validation and ownership. Final 10 minutes: discuss a database error and a useful test.

Assessment: look for correct data handling, basic API semantics, explicit authorization, and tests that check outcomes. A candidate who only lists decorators without understanding request flow needs more framework practice.

A mid level mock interview lasting seventy five minutes

First 15 minutes: Q084, Q090, Q120, and Q157. Next 20 minutes: solve Q217 or Q227. Next 25 minutes: discuss Q234 or Q236, focusing on retries and durable state. Final 15 minutes: investigate Q243 or Q246 and explain the evidence needed before changing configuration.

Assessment: the candidate should connect Python behavior to production resource limits, choose transaction boundaries, recognize duplicate effects, and diagnose a concrete failure instead of proposing unrelated tools.

A senior mock interview lasting ninety minutes

First 15 minutes: examine Q115, Q169, Q199, and Q212. Next 15 minutes: review an unsafe read-modify-write implementation and propose a correct alternative. Next 40 minutes: design Q235 or Q240 with explicit estimates, failure states, and deployment considerations. Final 20 minutes: discuss a migration, an incident, and a disagreement using real experience.

Assessment: look for assumptions, invariants, prioritization, trade-offs, and a validation plan. A senior candidate should acknowledge uncertainty and explain how to resolve it. Introducing microservices, queues, caches, and Kubernetes without a requirement is not by itself evidence of senior judgment.

A practical self assessment scale

Score 0 when you cannot explain the topic. Score 1 when you can repeat a definition but cannot apply it.

Score 2 when you can solve a normal example and explain its cost. Score 3 when you can handle boundary

and failure cases. Score 4 when you can compare alternatives, connect the topic to production behavior, and design a meaningful validation plan.

Record evidence beside the score: a solved exercise, a tested endpoint, a query plan, a failure simulation, or an explained design. Revisit low-scoring topics after a few days. Years of experience should not replace this evidence-based assessment.

Suggested portfolio projects by stage

Beginner project: a command-line expense or reading tracker with file persistence, explicit parsing errors, and tests. Focus on functions, collections, files, and code organization.

Junior project: an authenticated task API using one framework and a relational database. Include pagination, ownership checks, migrations, response schemas, and integration tests.

Mid-level project: a file-processing or source-ingestion service with durable storage, queued work, idempotency, retries, and observable progress. Demonstrate recovery after a worker crash.

Senior project: extend the same service with tenant isolation, capacity measurements, a safe schema migration, failure simulations, an SLO, and an architecture decision record. The strongest demonstration is an explained trade-off supported by evidence, not the largest collection of infrastructure logos.

Turning the guide into a blog series

Publish chapters in reading order or group related chapters into larger articles. Each article should start with who it helps and the problem it explains, then present questions with answers and practical examples. End with exercises or follow-ups that let readers test their understanding.

Keep the technical baseline visible. When updating a framework example, retest it against an explicit dependency set and revise the version note. Preserve the distinction between runnable code, application fragments, and architectural pseudocode. Avoid turning illustrative throughput assumptions into performance claims.

Useful series groupings are beginner Python, intermediate language behavior, coding patterns, concurrency and testing, HTTP and frameworks, databases and reliable jobs, production engineering, data and AI services, and senior system design. Readers can follow the full path or enter at the section that matches their current work.

25 Official references and further study

References support specific technical details and version notes; the practice questions, code exercises, interview scenarios, and illustrative designs are original explanations. Documentation pages can change, so use the version selector or pin dependencies when reproducing a framework example. Accessed September 15, 2026.

[1] Python 3.14 data model. Object protocols, descriptors, comparison behavior, and method lookup.
<https://docs.python.org/3.14/reference/datamodel.html>

[2] Python support for free threading. Build behavior, compatibility, and thread-safety considerations.
<https://docs.python.org/3/howto/free-threading-python.html>

[3] Python coroutines and tasks. Task groups, cancellation, deadlines, and awaitables.
<https://docs.python.org/3/library/asyncio-task.html>

[4] Python 3.14 multiprocessing. Start methods, process lifecycles, and portability.
<https://docs.python.org/3.14/library/multiprocessing.html>

- [5] SQLAlchemy 2.0 Session basics. Identity maps, transactions, flush behavior, and concurrency scope.
https://docs.sqlalchemy.org/en/20/orm/session_basics.html
- [6] NumPy broadcasting. Shape compatibility and memory implications.
<https://numpy.org/doc/stable/user/basics.broadcasting.html>
- [7] pandas Copy-on-Write. Version-specific assignment and copy behavior.
https://pandas.pydata.org/docs/user_guide/copy_on_write.html
- [8] Python Packaging User Guide. Project metadata and build configuration.
<https://packaging.python.org/en/latest/guides/writing-pyproject-toml/>
- [9] pytest fixtures. Dependency setup, fixture scopes, and teardown. <https://docs.pytest.org/en/stable/how-to/fixtures.html>
- [10] Django 5.2 database optimization. Query evaluation and relation loading.
<https://docs.djangoproject.com/en/5.2/topics/db/optimization/>
- [11] Django 5.2 asynchronous support. Async views, middleware, ORM boundaries, and transactions.
<https://docs.djangoproject.com/en/5.2/topics/async/>
- [12] Django REST Framework permissions. Object permissions and list/create limitations.
<https://www.django-rest-framework.org/api-guide/permissions/>
- [13] Flask async and await. WSGI execution behavior and background-task limitations.
<https://flask.palletsprojects.com/en/stable/async-await/>
- [14] FastAPI concurrency and async. Path function behavior and blocking helper calls.
<https://fastapi.tiangolo.com/async/>
- [15] FastAPI dependencies. Dependency declarations and reuse. <https://fastapi.tiangolo.com/tutorial/dependencies/>
- [16] Pydantic migration guide. Version 2 APIs and field requirements. <https://docs.pydantic.dev/latest/migration/>
- [17] Flask quickstart. Sessions, routing, contexts, and development-server behavior.
<https://flask.palletsprojects.com/en/stable/quickstart/>
- [18] PostgreSQL transaction isolation. Snapshot behavior and serialization failures.
<https://www.postgresql.org/docs/current/transaction-iso.html>
- [19] PostgreSQL multicolumn indexes. Index ordering and access strategies.
<https://www.postgresql.org/docs/current/indexes-multicolumn.html>
- [20] Celery tasks. Acknowledgements, retries, and task execution behavior.
<https://docs.celeryq.dev/en/stable/userguide/tasks.html>
- [21] FastAPI background tasks. In-process work and heavier task processing.
<https://fastapi.tiangolo.com/tutorial/background-tasks/>
- [22] Django 5.2 transactions. Atomic scopes and commit callbacks.
<https://docs.djangoproject.com/en/5.2/topics/db/transactions/>
- [23] SQLAlchemy 2.0 asyncio. AsyncSession ownership and implicit I/O concerns.
<https://docs.sqlalchemy.org/en/20/orm/extensions/asyncio.html>
- [24] Python 3.14 annotationlib. Annotation introspection and evaluation behavior.
<https://docs.python.org/3.14/library/annotationlib.html>
- [25] What is new in Python 3.14. Version-specific language and runtime changes.
<https://docs.python.org/3.14/whatsnew/3.14.html>